



Código Fonte Completo

Sistema de Gestão para Restaurantes e Bares

JavaFX + Hibernate + SQLite/PostgreSQL

Corumbá Sistemas

Janeiro 2026

Desenvolvido por: Corumbá Sistemas
77 arquivos de código fonte (JDK 17 + JDK 21)

1. Configuracao e Build (pom.xml)

pom.xml

pom.xml • 60 linhas

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <groupId>com.corumbasistemas</groupId>
  <artifactId>coruba-food</artifactId>
  <version>2.0.0</version>
  <packaging>jar</packaging>
  <name>Corumbá Food - MesaPronta</name>

  <properties>
    <maven.compiler.source>17</maven.compiler.source>
    <maven.compiler.target>17</maven.compiler.target>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
    <javafx.version>21.0.8</javafx.version>
    <hibernate.version>6.4.4.Final</hibernate.version>
    <sqlite.version>3.45.1.0</sqlite.version>
    <jbcrypt.version>0.4</jbcrypt.version>
    <logback.version>1.5.3</logback.version>
  </properties>

  <dependencies>
    <dependency><groupId>org.openjfx</groupId><artifactId>javafx-controls</artifactId><version>${javafx.version}</version></dependency>
    <dependency><groupId>org.openjfx</groupId><artifactId>javafx-fxml</artifactId><version>${javafx.version}</version></dependency>
    <dependency><groupId>org.openjfx</groupId><artifactId>javafx-graphics</artifactId><version>${javafx.version}</version></dependency>
    <dependency><groupId>org.hibernate.orm</groupId><artifactId>hibernate-core</artifactId><version>${hibernate.version}</version></
  </dependency>
    <dependency><groupId>org.hibernate.orm</groupId><artifactId>hibernate-c3p0</artifactId><version>${hibernate.version}</version></
  </dependency>
    <dependency><groupId>org.xerial</groupId><artifactId>sqlite-jdbc</artifactId><version>${sqlite.version}</version></dependency>
    <dependency><groupId>org.hibernate.orm</groupId><artifactId>hibernate-community-dialects</artifactId><version>${hibernate.version}</
  </version></dependency>
    <dependency><groupId>org.mindrot</groupId><artifactId>jbcrypt</artifactId><version>${jbcrypt.version}</version></dependency>
    <dependency><groupId>ch.qos.logback</groupId><artifactId>logback-classic</artifactId><version>${logback.version}</version></
  </dependency>
    <dependency><groupId>org.slf4j</groupId><artifactId>slf4j-api</artifactId><version>2.0.12</version></dependency>
    <dependency><groupId>com.fasterxml.jackson.core</groupId><artifactId>jackson-databind</artifactId><version>2.17.0</version></
  </dependency>
    <dependency><groupId>org.junit.jupiter</groupId><artifactId>junit-jupiter</artifactId><version>5.10.2</version><scope>test</scope></
  </dependency>
  </dependencies>

  <build>
    <plugins>
      <plugin>
        <groupId>org.apache.maven.plugins</groupId>
        <artifactId>maven-compiler-plugin</artifactId>
        <version>3.12.1</version>
        <configuration>
          <source>17</source>
          <target>17</target>
        </configuration>
      </plugin>
      <plugin>
        <groupId>org.openjfx</groupId>
        <artifactId>javafx-maven-plugin</artifactId>
        <version>0.0.8</version>
        <configuration>
          <mainClass>com.corumbasistemas.corubafood.CorubaFoodApp</mainClass>
        </configuration>
      </plugin>
    </plugins>
  </build>
</project>
```

pom.xml

jdk21/pom.xml • 1 linhas

[ARQUIVO NAO ENCONTRADO: jdk21/pom.xml]

2. Aplicacao Principal

```

CorubaFoodApp.java src/main/java/com/corumbasistemas/corubafood/CorubaFoodApp.java • 136 linhas

package com.corumbasistemas.corubafood;

import com.corumbasistemas.corubafood.controller.AuthController;
import com.corumbasistemas.corubafood.model.Usuario;
import com.corumbasistemas.corubafood.service.SyncService;
import com.corumbasistemas.corubafood.util.*;
import javafx.application.Application;
import javafx.fxml.FXMLLoader;
import javafx.scene.*;
import javafx.stage.Stage;
import org.slf4j.*;

/**
 * CorubaFoodApp - MesaPronta v2.0
 * JDK 17+ com JavaFX
 *
 * Login: CNPJ "12.345.678/0001-90", usuario "admin", senha "admin123"
 */
public class CorubaFoodApp extends Application {
    private static final Logger logger = LoggerFactory.getLogger(CorubaFoodApp.class);
    private static Stage ps;
    private static AuthController ac;
    private static Usuario ul;
    private static SyncService syncService;
    private static Thread syncThread;

    @Override
    public void start(Stage s) throws Exception {
        ps = s;
        ac = new AuthController();
        syncService = new SyncService();
        JPAUtil.getEMFCliente();
        logger.info("Coruba Food - MesaPronta v2.0 iniciando...");

        try {
            SeedData.popular();
        } catch (Exception e) {
            logger.warn("Seed: " + e.getMessage());
        }

        iniciarSyncBackground();
        mostrarLogin();
    }

    private void iniciarSyncBackground() {
        syncThread = new Thread(() -> {
            while (true) {
                try {
                    Thread.sleep(5 * 60 * 1000);
                    if (ul != null) {
                        logger.info("Sync automatico iniciando...");
                        syncService.sincronizarTudo();
                    }
                } catch (InterruptedException e) {
                    break;
                } catch (Exception e) {
                    logger.error("Sync auto erro: " + e.getMessage());
                }
            }
        });
        syncThread.setDaemon(true);
        syncThread.setName("SyncThread");
        syncThread.start();
        logger.info("Thread de sync iniciada");
    }

    public static void mostrarLogin() throws Exception {
        var l = new FXMLLoader(CorubaFoodApp.class.getResource("/view/LoginView.fxml"));
        Parent r = l.load();
        ps.setTitle("Coruba Food - Login");
        ps.setScene(new Scene(r));
        ps.setResizable(false);
        ps.show();
    }

    public static void mostrarPrincipal() throws Exception {
        var l = new FXMLLoader(CorubaFoodApp.class.getResource("/view/PrincipalView.fxml"));
        Parent r = l.load();
        String titulo = "Coruba Food - MesaPronta [%s]".formatted(ul.getNome());
        ps.setTitle(titulo);
        ps.setScene(new Scene(r, 1200, 800));
        ps.setResizable(true);
        ps.setMaximized(true);
        new Thread(() -> syncService.sincronizarTudo()).start();
    }
}

```

```

}

public static void mostrarDelivery() throws Exception {
    var l = new FXMLLoader(CorubaFoodApp.class.getResource("/view/DeliveryView.fxml"));
    Parent r = l.load();
    ps.setTitle("Coruba Food - Delivery [%s]".formatted(ul.getNome()));
    ps.setScene(new Scene(r, 1200, 800));
    ps.setResizable(true);
    ps.setMaximized(true);
    ps.show();
}

public static void mostrarConfigIntegracao() throws Exception {
    var l = new FXMLLoader(CorubaFoodApp.class.getResource("/view/ConfigIntegracaoView.fxml"));
    Parent r = l.load();
    ps.setTitle("Coruba Food - Configuracao APIs [" + ul.getNome() + "]");
    ps.setScene(new Scene(r, 700, 700));
    ps.setResizable(true);
    ps.show();
}

public static Stage getPS() { return ps; }
public static AuthController getAuthController() { return ac; }
public static Usuario getUsuarioLogado() { return ul; }
public static void setUsuarioLogado(Usuario u) { ul = u; }
public static SyncService getSyncService() { return syncService; }

@Override
public void stop() {
    logger.info("Encerrando Coruba Food...");
    try {
        syncService.sincronizarTudo();
    } catch (Exception e) {
        logger.warn("Sync final falhou: " + e.getMessage());
    }
    JPAUtil.shutdown();
    logger.info("Coruba Food parado");
}

public static void main(String... args) {
    try {
        launch(args);
    } catch (IllegalStateException e) {
        System.err.println("JavaFX Runtime already started: " + e.getMessage());
    } catch (Exception e) {
        System.err.println("Erro: " + e.getMessage());
        e.printStackTrace();
        System.exit(1);
    }
}
}

```

 **CorubaFoodApp.java**

jdk21/src/main/java/com/corumbasistemas/corubafood/CorubaFoodApp.java • 1 linhas

[ARQUIVO NAO ENCONTRADO: jdk21/src/main/java/com/corumbasistemas/corubafood/CorubaFoodApp.java]

3. Models (Entidades JPA)

 **Usuario.java**

src/main/java/com/corumbasistemas/corubafood/model/Usuario.java • 91 linhas

```
package com.corumbasistemas.corubafood.model;

import jakarta.persistence.*;
import java.time.LocalDateTime;

/**
 * Usuario - BANCO CLIENTE (SQLite)
 * Guarda credenciais de acesso
 */
@Entity
@Table(name = "usuario_cliente")
public class Usuario {

    public enum Papel { ADMIN, GERENTE, GARCOM, CAIXA, COZINHA }

    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, length = 100)
    private String nome;

    @Column(nullable = false, length = 50, unique = true)
    private String username;

    @Column(nullable = false, length = 200)
    private String senha; // BCrypt hash

    @Enumerated(EnumType.STRING)
    @Column(nullable = false, length = 20)
    private Papel papel;

    // CNPJ da empresa (referencia ao banco cliente)
    @Column(nullable = false, length = 18)
    private String empresaDocumento;

    @Column(nullable = false)
    private Boolean ativo = true;

    @Column(name = "created_at")
    private LocalDateTime createdAt;

    // Ambiente: cloud ou local
    @Column(length = 10)
    private String ambiente = "local";

    @PrePersist
    protected void onCreate() {
        createdAt = LocalDateTime.now();
        if (ambiente == null) ambiente = "local";
    }

    // Construtor padrão
    public Usuario() {}

    // Construtor
    public Usuario(String nome, String username, String senha, Papel papel, String empresaDocumento) {
        this.nome = nome;
        this.username = username;
        this.senha = senha;
        this.papel = papel;
        this.empresaDocumento = empresaDocumento;
        this.ativo = true;
        this.ambiente = "local";
    }

    // Getters/Setters
    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }
    public String getNome() { return nome; }
    public void setNome(String nome) { this.nome = nome; }
    public String getUsername() { return username; }
    public void setUsername(String username) { this.username = username; }
    public String getSenha() { return senha; }
    public void setSenha(String senha) { this.senha = senha; }
    public Papel getPapel() { return papel; }
    public void setPapel(Papel papel) { this.papel = papel; }
    public String getEmpresaDocumento() { return empresaDocumento; }
    public void setEmpresaDocumento(String empresaDocumento) { this.empresaDocumento = empresaDocumento; }
    public Boolean getAtivo() { return ativo; }
    public void setAtivo(Boolean ativo) { this.ativo = ativo; }
    public LocalDateTime getCreatedAt() { return createdAt; }
    public void setCreatedAt(LocalDateTime createdAt) { this.createdAt = createdAt; }
    public String getAmbiente() { return ambiente; }
    public void setAmbiente(String ambiente) { this.ambiente = ambiente; }
```

```

@Override
public String toString() {
    return "%s (%s) - %s".formatted(nome, username, papel);
}
}

```

Produto.java

src/main/java/com/corumbasistemas/corubafood/model/Produto.java • 84 linhas

```

package com.corumbasistemas.corubafood.model;

import jakarta.persistence.*;
import java.math.BigDecimal;
import java.time.LocalDateTime;

/**
 * Produto - BANCO NUVEM (PostgreSQL)
 * Cardápio completo do restaurante
 */
@Entity
@Table(name = "produto_nuvem", uniqueConstraints = {
    @UniqueConstraint(columnNames = {"codigo", "empresa_documento"}, name = "uk_produto_cod_emp")
})
public class Produto {

    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, length = 20)
    private String codigo;

    @Column(nullable = false, length = 200)
    private String nome;

    @Column(length = 500)
    private String descricao;

    @Column(nullable = false, precision = 10, scale = 2)
    private BigDecimal preco;

    @Column(nullable = false, length = 18)
    private String empresaDocumento; // CNPJ da empresa dona

    @Column(length = 18)
    private String categoriaDocumento;

    @Column(nullable = false)
    private Boolean ativo = true;

    @Column(name = "created_at")
    private LocalDateTime createdAt;

    @PrePersist
    protected void onCreate() {
        createdAt = LocalDateTime.now();
    }

    public Produto() {}

    public Produto(String codigo, String nome, BigDecimal preco, String empresaDocumento) {
        this.codigo = codigo;
        this.nome = nome;
        this.preco = preco;
        this.empresaDocumento = empresaDocumento;
        this.ativo = true;
    }

    // Getters/Setters
    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }
    public String getCodigo() { return codigo; }
    public void setCodigo(String codigo) { this.codigo = codigo; }
    public String getNome() { return nome; }
    public void setNome(String nome) { this.nome = nome; }
    public String getDescricao() { return descricao; }
    public void setDescricao(String descricao) { this.descricao = descricao; }
    public BigDecimal getPreco() { return preco; }
    public void setPreco(BigDecimal preco) { this.preco = preco; }
    public String getEmpresaDocumento() { return empresaDocumento; }
    public void setEmpresaDocumento(String empresaDocumento) { this.empresaDocumento = empresaDocumento; }
    public String getCategoriaDocumento() { return categoriaDocumento; }
    public void setCategoriaDocumento(String categoriaDocumento) { this.categoriaDocumento = categoriaDocumento; }
    public Boolean getAtivo() { return ativo; }
    public void setAtivo(Boolean ativo) { this.ativo = ativo; }
    public LocalDateTime getCreatedAt() { return createdAt; }
    public void setCreatedAt(LocalDateTime createdAt) { this.createdAt = createdAt; }

    @Override
    public String toString() {
        return "%s - R$ %s".formatted(nome, preco);
    }
}

```

```
}
}
```

Categoria.java

src/main/java/com/corumbasistemas/corubafood/model/Categoria.java • 27 linhas

```
package com.corumbasistemas.corubafood.model;

import jakarta.persistence.*;
import java.time.LocalDateTime;

@Entity
@Table(name = "categoria", uniqueConstraints = {
    @UniqueConstraint(columnNames = {"nome", "empresa_id"}, name = "uk_categoria_nome_empresa")
})
public class Categoria {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY) private Long id;
    @Column(nullable = false, length = 100) private String nome;
    @Column(length = 255) private String descricao;
    @Column(name = "ordem") private Integer ordem = 0;
    @ManyToOne(fetch = FetchType.LAZY) @JoinColumn(name = "empresa_id", nullable = false) private Empresa empresa;
    @Column(name = "ativo", nullable = false) private Boolean ativo = true;
    @Column(name = "created_at") private LocalDateTime createdAt;
    @PrePersist protected void onCreate() { createdAt = LocalDateTime.now(); }
    public Long getId() { return id; } public void setId(Long id) { this.id = id; }
    public String getNome() { return nome; } public void setNome(String n) { this.nome = n; }
    public String getDescricao() { return descricao; } public void setDescricao(String d) { this.descricao = d; }
    public Integer getOrdem() { return ordem; } public void setOrdem(Integer o) { this.ordem = o; }
    public Empresa getEmpresa() { return empresa; } public void setEmpresa(Empresa e) { this.empresa = e; }
    public Boolean getAtivo() { return ativo; } public void setAtivo(Boolean a) { this.ativo = a; }
    public LocalDateTime getCreatedAt() { return createdAt; } public void setCreatedAt(LocalDateTime c) { this.createdAt = c; }
}
```

Mesa.java

src/main/java/com/corumbasistemas/corubafood/model/Mesa.java • 79 linhas

```
package com.corumbasistemas.corubafood.model;

import jakarta.persistence.*;
import java.time.LocalDateTime;

/**
 * Mesa - BANCO NUVEM (PostgreSQL)
 * Mesas do restaurante, identificadas pelo CNPJ da empresa
 */
@Entity
@Table(name = "mesa_nuvem", uniqueConstraints = {
    @UniqueConstraint(columnNames = {"numero", "empresa_documento"}, name = "uk_mesa_num_emp")
})
public class Mesa {

    public enum Status { LIVRE, OCUPADA, RESERVADA }

    public String getDescricao() {
        return switch (this.status) {
            case LIVRE -> "Disponível";
            case OCUPADA -> "Em uso";
            case RESERVADA -> "Reservada";
        };
    }

    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false)
    private Integer numero;

    @Column(nullable = false)
    private Integer capacidade;

    @Enumerated(EnumType.STRING)
    @Column(nullable = false, length = 20)
    private Status status = Status.LIVRE;

    // CNPJ da empresa (banco nuvem não tem FK para banco cliente)
    @Column(nullable = false, length = 18, name = "empresa_documento")
    private String empresaDocumento;

    @Column(name = "created_at")
    private LocalDateTime createdAt;

    @PrePersist
    protected void onCreate() {
        createdAt = LocalDateTime.now();
    }

    public Mesa() {}

    public Mesa(Integer numero, Integer capacidade, String empresaDocumento) {
        this.numero = numero;
    }
}
```

```

        this.capacidade = capacidade;
        this.empresaDocumento = empresaDocumento;
        this.status = Status.LIVRE;
    }

    // Getters/Setters
    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }
    public Integer getNumero() { return numero; }
    public void setNumero(Integer numero) { this.numero = numero; }
    public Integer getCapacidade() { return capacidade; }
    public void setCapacidade(Integer capacidade) { this.capacidade = capacidade; }
    public Status getStatus() { return status; }
    public void setStatus(Status status) { this.status = status; }
    public String getEmpresaDocumento() { return empresaDocumento; }
    public void setEmpresaDocumento(String empresaDocumento) { this.empresaDocumento = empresaDocumento; }
    public LocalDateTime getCreatedAt() { return createdAt; }
    public void setCreatedAt(LocalDateTime createdAt) { this.createdAt = createdAt; }

    @Override
    public String toString() {
        return "Mesa %d (%d lugares) - %s".formatted(numero, capacidade, status);
    }
}

```

Pedido.java

src/main/java/com/corumbasistemas/corubafood/model/Pedido.java • 76 linhas

```

package com.corumbasistemas.corubafood.model;

import jakarta.persistence.*;
import java.math.BigDecimal;
import java.time.LocalDateTime;
import java.util.ArrayList;
import java.util.List;

/**
 * Pedido - BANCO NUVEM (PostgreSQL)
 * Pedidos do restaurante, identificados pelo CNPJ da empresa
 */
@Entity
@Table(name = "pedido_nuvem")
public class Pedido {

    public enum Status { ABERTO, EM_PREPARO, PRONTO, ENTREGUE, CANCELADO }

    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    // IDs de referencia (sem FK para outros bancos)
    @Column(name = "mesa_id")
    private Long mesaId;

    @Column(name = "usuario_id")
    private Long usuarioId;

    // CNPJ da empresa (chave de sincronizacao)
    @Column(nullable = false, length = 18, name = "empresa_documento")
    private String empresaDocumento;

    @Enumerated(EnumType.STRING)
    @Column(nullable = false, length = 20)
    private Status status = Status.ABERTO;

    @Column(precision = 10, scale = 2)
    private BigDecimal total = BigDecimal.ZERO;

    @Column(name = "created_at")
    private LocalDateTime createdAt;

    @OneToMany(mappedBy = "pedido", cascade = CascadeType.ALL)
    private List<ItemPedido> itens = new ArrayList<>();

    @PrePersist
    protected void onCreate() {
        createdAt = LocalDateTime.now();
    }

    public Pedido() {}

    // Getters/Setters
    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }
    public Long getMesaId() { return mesaId; }
    public void setMesaId(Long mesaId) { this.mesaId = mesaId; }
    public Long getUsuarioId() { return usuarioId; }
    public void setUsuarioId(Long usuarioId) { this.usuarioId = usuarioId; }
    public String getEmpresaDocumento() { return empresaDocumento; }
    public void setEmpresaDocumento(String empresaDocumento) { this.empresaDocumento = empresaDocumento; }
    public Status getStatus() { return status; }
    public void setStatus(Status status) { this.status = status; }
    public BigDecimal getTotal() { return total; }
    public void setTotal(BigDecimal total) { this.total = total; }
}

```



```

    public LocalDateTime getCreatedAt() { return createdAt; }
    public void setCreatedAt(LocalDateTime createdAt) { this.createdAt = createdAt; }
    public List<ItemPedido> getItens() { return itens; }
    public void setItens(List<ItemPedido> itens) { this.itens = itens; }

    @Override
    public String toString() {
        return "Pedido #%- %s - R$ %s".formatted(id, status, total);
    }
}

```

ItemPedido.java

src/main/java/com/corumbasistemas/corubafood/model/ItemPedido.java • 54 linhas

```

package com.corumbasistemas.corubafood.model;

import jakarta.persistence.*;
import java.math.BigDecimal;

/**
 * ItemPedido - BANCO NUVEM (PostgreSQL)
 */
@Entity
@Table(name = "item_pedido_nuvem")
public class ItemPedido {

    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "pedido_id", nullable = false)
    private Pedido pedido;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "produto_id", nullable = false)
    private Produto produto;

    @Column(nullable = false)
    private Integer quantidade;

    @Column(nullable = false, precision = 10, scale = 2)
    private BigDecimal precoUnitario;

    @Column(nullable = false, precision = 10, scale = 2)
    private BigDecimal subtotal;

    @Column(length = 18, name = "empresa_documento")
    private String empresaDocumento;

    public ItemPedido() {}

    // Getters/Setters
    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }
    public Pedido getPedido() { return pedido; }
    public void setPedido(Pedido pedido) { this.pedido = pedido; }
    public Produto getProduto() { return produto; }
    public void setProduto(Produto produto) { this.produto = produto; }
    public Integer getQuantidade() { return quantidade; }
    public void setQuantidade(Integer quantidade) { this.quantidade = quantidade; }
    public BigDecimal getPrecoUnitario() { return precoUnitario; }
    public void setPrecoUnitario(BigDecimal precoUnitario) { this.precoUnitario = precoUnitario; }
    public BigDecimal getSubtotal() { return subtotal; }
    public void setSubtotal(BigDecimal subtotal) { this.subtotal = subtotal; }
    public String getEmpresaDocumento() { return empresaDocumento; }
    public void setEmpresaDocumento(String empresaDocumento) { this.empresaDocumento = empresaDocumento; }
}

```

Pagamento.java

src/main/java/com/corumbasistemas/corubafood/model/Pagamento.java • 29 linhas

```

package com.corumbasistemas.corubafood.model;

import jakarta.persistence.*;
import java.math.BigDecimal;
import java.time.LocalDateTime;

@Entity
@Table(name = "pagamento")
public class Pagamento {

    public enum FormaPagamento { DINHEIRO, CARTAO_CREDITO, CARTAO_DEBITO, PIX, VALE_REFEICAO }
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY) private Long id;
    @ManyToOne(fetch = FetchType.LAZY) @JoinColumn(name = "pedido_id", nullable = false) private Pedido pedido;
    @ManyToOne(fetch = FetchType.LAZY) @JoinColumn(name = "empresa_id", nullable = false) private Empresa empresa;
    @Enumerated(EnumType.STRING) @Column(nullable = false, length = 30) private FormaPagamento formaPagamento;
    @Column(nullable = false, precision = 10, scale = 2) private BigDecimal valor;
    @Column(precision = 10, scale = 2) private BigDecimal desconto = BigDecimal.ZERO;
    @Column(name = "desconto_autorizado_por", length = 255) private String descontoAutorizadoPor;
    @Column(name = "created_at") private LocalDateTime createdAt;
    @PrePersist protected void onCreate() { createdAt = LocalDateTime.now(); }
}

```

```

    public Long getId() { return id; } public void setId(Long id) { this.id = id; }
    public Pedido getPedido() { return pedido; } public void setPedido(Pedido p) { this.pedido = p; }
    public Empresa getEmpresa() { return empresa; } public void setEmpresa(Empresa e) { this.empresa = e; }
    public FormaPagamento getFormaPagamento() { return formaPagamento; } public void setFormaPagamento(FormaPagamento f) {
this.formaPagamento = f; }
    public BigDecimal getValor() { return valor; } public void setValor(BigDecimal v) { this.valor = v; }
    public BigDecimal getDesconto() { return desconto; } public void setDesconto(BigDecimal d) { this.desconto = d; }
    public String getDescontoAutorizadoPor() { return descontoAutorizadoPor; } public void setDescontoAutorizadoPor(String d) {
this.descontoAutorizadoPor = d; }
    public LocalDateTime getCreatedAt() { return createdAt; } public void setCreatedAt(LocalDateTime c) { this.createdAt = c; }
}

```



PedidoDelivery.java

src/main/java/com/corumbasistemas/corubafood/model/PedidoDelivery.java • 167 linhas

```

package com.corumbasistemas.corubafood.model;

import jakarta.persistence.*;
import java.math.BigDecimal;
import java.time.LocalDateTime;
import java.util.ArrayList;
import java.util.List;

/**
 * PedidoDelivery - Pedidos vindos de plataformas de delivery (iFood, 99Food).
 */
@Entity
@Table(name = "pedidos_delivery")
public class PedidoDelivery {

    public enum Plataforma { IFOOD, FOOD99 }
    public enum StatusPedido { NOVO, CONFIRMADO, PREPARANDO, PRONTO, SAIU_ENTREGA, ENTREGUE, CANCELADO }

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(name = "codigo_plataforma", nullable = false, length = 50)
    private String codigoPlataforma;

    @Enumerated(EnumType.STRING)
    @Column(nullable = false, length = 20)
    private Plataforma plataforma;

    @Column(name = "nome_cliente", nullable = false, length = 200)
    private String nomeCliente;

    @Column(name = "telefone_cliente", length = 20)
    private String telefoneCliente;

    @Column(name = "endereco_entrega", length = 500)
    private String enderecoEntrega;

    @OneToMany(cascade = CascadeType.ALL, orphanRemoval = true)
    @JoinColumn(name = "pedido_delivery_id")
    private List<ItemPedidoDelivery> itens = new ArrayList<>();

    @Column(name = "valor_itens", nullable = false, precision = 10, scale = 2)
    private BigDecimal valorItens;

    @Column(name = "taxa_entrega", precision = 10, scale = 2)
    private BigDecimal taxaEntrega = BigDecimal.ZERO;

    @Column(name = "valor_total", nullable = false, precision = 10, scale = 2)
    private BigDecimal valorTotal;

    @Enumerated(EnumType.STRING)
    @Column(nullable = false, length = 20)
    private StatusPedido status = StatusPedido.NOVO;

    @Column(name = "forma_pagamento", length = 50)
    private String formaPagamento;

    @Column(name = "observacoes", length = 500)
    private String observacoes;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "empresa_id", nullable = false)
    private Empresa empresa;

    @Column(name = "data_criacao", nullable = false)
    private LocalDateTime dataCriacao = LocalDateTime.now();

    @Column(name = "data_confirmacao")
    private LocalDateTime dataConfirmacao;

    @Column(name = "data_pronto")
    private LocalDateTime dataPronto;

    @Column(name = "data_entrega")
    private LocalDateTime dataEntrega;

    @Column(name = "data_cancelamento")

```

```

private LocalDateTime dataCancelamento;

@Column(name = "motivo_cancelamento", length = 500)
private String motivoCancelamento;

public PedidoDelivery() {}

public PedidoDelivery(Plataforma plataforma, String codigoPlataforma, String nomeCliente, Empresa empresa) {
    this.plataforma = plataforma;
    this.codigoPlataforma = codigoPlataforma;
    this.nomeCliente = nomeCliente;
    this.empresa = empresa;
    this.valorItens = BigDecimal.ZERO;
    this.valorTotal = BigDecimal.ZERO;
    this.status = StatusPedido.NOVO;
}

public void calcularTotal() {
    if (itens == null || itens.isEmpty()) {
        this.valorItens = BigDecimal.ZERO;
    } else {
        this.valorItens = itens.stream()
            .map(ItemPedidoDelivery::getSubtotal)
            .reduce(BigDecimal.ZERO, BigDecimal::add);
    }
    BigDecimal taxa = this.taxaEntrega != null ? this.taxaEntrega : BigDecimal.ZERO;
    this.valorTotal = this.valorItens.add(taxa);
}

public boolean avancarStatus() {
    switch (this.status) {
        case NOVO: this.status = StatusPedido.CONFIRMADO; this.dataConfirmacao = LocalDateTime.now(); return true;
        case CONFIRMADO: this.status = StatusPedido.PREPARANDO; return true;
        case PREPARANDO: this.status = StatusPedido.PRONTO; this.dataPronto = LocalDateTime.now(); return true;
        case PRONTO: this.status = StatusPedido.SAIU_ENTREGA; return true;
        case SAIU_ENTREGA: this.status = StatusPedido.ENTREGUE; this.dataEntrega = LocalDateTime.now(); return true;
        default: return false;
    }
}

public void cancelar(String motivo) {
    this.status = StatusPedido.CANCELADO;
    this.motivoCancelamento = motivo;
    this.dataCancelamento = LocalDateTime.now();
}

// Getters e Setters
public Long getId() { return id; }
public void setId(Long id) { this.id = id; }
public String getCodigoPlataforma() { return codigoPlataforma; }
public void setCodigoPlataforma(String c) { this.codigoPlataforma = c; }
public Plataforma getPlataforma() { return plataforma; }
public void setPlataforma(Plataforma p) { this.plataforma = p; }
public String getNomeCliente() { return nomeCliente; }
public void setNomeCliente(String n) { this.nomeCliente = n; }
public String getTelefoneCliente() { return telefoneCliente; }
public void setTelefoneCliente(String t) { this.telefoneCliente = t; }
public String getEnderecoEntrega() { return enderecoEntrega; }
public void setEnderecoEntrega(String e) { this.enderecoEntrega = e; }
public List<ItemPedidoDelivery> getItens() { return itens; }
public void setItens(List<ItemPedidoDelivery> i) { this.itens = i; }
public BigDecimal getValorItens() { return valorItens; }
public void setValorItens(BigDecimal v) { this.valorItens = v; }
public BigDecimal getTaxaEntrega() { return taxaEntrega; }
public void setTaxaEntrega(BigDecimal t) { this.taxaEntrega = t; }
public BigDecimal getValorTotal() { return valorTotal; }
public void setValorTotal(BigDecimal v) { this.valorTotal = v; }
public StatusPedido getStatus() { return status; }
public void setStatus(StatusPedido s) { this.status = s; }
public String getFormaPagamento() { return formaPagamento; }
public void setFormaPagamento(String f) { this.formaPagamento = f; }
public String getObservacoes() { return observacoes; }
public void setObservacoes(String o) { this.observacoes = o; }
public Empresa getEmpresa() { return empresa; }
public void setEmpresa(Empresa e) { this.empresa = e; }
public LocalDateTime getDataCriacao() { return dataCriacao; }
public void setDataCriacao(LocalDateTime d) { this.dataCriacao = d; }
public LocalDateTime getDataConfirmacao() { return dataConfirmacao; }
public void setDataConfirmacao(LocalDateTime d) { this.dataConfirmacao = d; }
public LocalDateTime getDataPronto() { return dataPronto; }
public void setDataPronto(LocalDateTime d) { this.dataPronto = d; }
public LocalDateTime getDataEntrega() { return dataEntrega; }
public void setDataEntrega(LocalDateTime d) { this.dataEntrega = d; }
public LocalDateTime getDataCancelamento() { return dataCancelamento; }
public void setDataCancelamento(LocalDateTime d) { this.dataCancelamento = d; }
public String getMotivoCancelamento() { return motivoCancelamento; }
public void setMotivoCancelamento(String m) { this.motivoCancelamento = m; }
}

```

```

package com.corumbasistemas.corubafood.model;

import jakarta.persistence.*;
import java.math.BigDecimal;

/**
 * ItemPedidoDelivery - Item individual de um pedido de delivery.
 *
 * Cada item representa um produto do cardápio que foi
 * pedido através do iFood ou 99Food.
 */
@Entity
@Table(name = "itens_pedido_delivery")
public class ItemPedidoDelivery {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    // Nome do produto no momento do pedido (pode mudar no cardápio depois)
    @Column(name = "nome_produto", nullable = false, length = 200)
    private String nomeProduto;

    // Quantidade pedida
    @Column(nullable = false)
    private Integer quantidade;

    // Preço unitário no momento do pedido
    @Column(name = "preco_unitario", nullable = false, precision = 10, scale = 2)
    private BigDecimal precoUnitario;

    // Subtotal = quantidade × preço unitário
    @Column(nullable = false, precision = 10, scale = 2)
    private BigDecimal subtotal;

    // Observações específicas do item (ex: "sem cebola", "mal passado")
    @Column(name = "observacoes", length = 300)
    private String observacoes;

    // Combo/adicionais
    @Column(name = "adicionais", length = 500)
    private String adicionais;

    // Construtor padrão
    public ItemPedidoDelivery() {}

    // Construtor com campos obrigatórios
    public ItemPedidoDelivery(String nomeProduto, Integer quantidade, BigDecimal precoUnitario) {
        this.nomeProduto = nomeProduto;
        this.quantidade = quantidade;
        this.precoUnitario = precoUnitario;
        calcularSubtotal();
    }

    /**
     * Calcula o subtotal do item
     */
    public void calcularSubtotal() {
        if (this.precoUnitario != null && this.quantidade != null) {
            this.subtotal = this.precoUnitario.multiply(BigDecimal.valueOf(this.quantidade));
        }
    }

    // --- Getters e Setters ---

    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }

    public String getNomeProduto() { return nomeProduto; }
    public void setNomeProduto(String nomeProduto) { this.nomeProduto = nomeProduto; }

    public Integer getQuantidade() { return quantidade; }
    public void setQuantidade(Integer quantidade) {
        this.quantidade = quantidade;
        calcularSubtotal();
    }

    public BigDecimal getPrecoUnitario() { return precoUnitario; }
    public void setPrecoUnitario(BigDecimal precoUnitario) {
        this.precoUnitario = precoUnitario;
        calcularSubtotal();
    }

    public BigDecimal getSubtotal() { return subtotal; }
    public void setSubtotal(BigDecimal subtotal) { this.subtotal = subtotal; }

    public String getObservacoes() { return observacoes; }
    public void setObservacoes(String observacoes) { this.observacoes = observacoes; }

    public String getAdicionais() { return adicionais; }
    public void setAdicionais(String adicionais) { this.adicionais = adicionais; }

    @Override
    public String toString() {
        return quantidade + "x " + nomeProduto + " = R$ " + subtotal;
    }
}

```

```

    }
}

```

Empresa.java

src/main/java/com/corumbasistemas/corubafood/model/Empresa.java • 102 linhas

```

package com.corumbasistemas.corubafood.model;

import jakarta.persistence.*;
import java.time.LocalDateTime;

/**
 * Empresa - BANCO CLIENTE (SQLite)
 * Guarda dados de autenticação e identificação
 */
@Entity
@Table(name = "empresa_cliente", uniqueConstraints = {
    @UniqueConstraint(columnNames = "documento", name = "uk_empresa_doc")
})
public class Empresa {

    public enum Tipo { RESTAURANTE, PIZZARIA, HAMBURGUERIA, CAFETERIA, BAR, OUTRO }

    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, length = 200)
    private String nome;

    @Column(nullable = false, length = 18, unique = true)
    private String documento; // CNPJ ou CPF

    @Enumerated(EnumType.STRING)
    @Column(length = 20)
    private Tipo tipo = Tipo.RESTAURANTE;

    @Column(length = 20)
    private String telefone;

    @Column(length = 300)
    private String endereco;

    @Column(length = 100)
    private String cidade;

    @Column(length = 2)
    private String estado;

    @Column(nullable = false)
    private Boolean ativo = true;

    // Credenciais da API da nuvem
    @Column(length = 500)
    private String apiUrl;

    @Column(length = 200)
    private String apiKey;

    @Column(name = "created_at")
    private LocalDateTime createdAt;

    @PrePersist
    protected void onCreate() {
        createdAt = LocalDateTime.now();
    }

    // Construtor padrão
    public Empresa() {}

    // Construtor
    public Empresa(String nome, String documento) {
        this.nome = nome;
        this.documento = documento;
        this.ativo = true;
    }

    // Getters/Setters
    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }
    public String getNome() { return nome; }
    public void setNome(String nome) { this.nome = nome; }
    public String getDocumento() { return documento; }
    public void setDocumento(String documento) { this.documento = documento; }
    public Tipo getTipo() { return tipo; }
    public void setTipo(Tipo tipo) { this.tipo = tipo; }
    public String getTelefone() { return telefone; }
    public void setTelefone(String telefone) { this.telefone = telefone; }
    public String getEndereco() { return endereco; }
    public void setEndereco(String endereco) { this.endereco = endereco; }
    public String getCidade() { return cidade; }
    public void setCidade(String cidade) { this.cidade = cidade; }
    public String getEstado() { return estado; }
    public void setEstado(String estado) { this.estado = estado; }

```

```

public Boolean getAtivo() { return ativo; }
public void setAtivo(Boolean ativo) { this.ativo = ativo; }
public String getApiUrl() { return apiUrl; }
public void setApiUrl(String apiUrl) { this.apiUrl = apiUrl; }
public String getApiKey() { return apiKey; }
public void setApiKey(String apiKey) { this.apiKey = apiKey; }
public LocalDateTime getCreatedAt() { return createdAt; }
public void setCreatedAt(LocalDateTime createdAt) { this.createdAt = createdAt; }

@Override
public String toString() {
    return "%s (%s)".formatted(nome, documento);
}
}

```

**Usuario.java**

jdk21/src/main/java/com/corumbasistemas/corubafood/model/Usuario.java • 1 linhas

[ARQUIVO NAO ENCONTRADO: jdk21/src/main/java/com/corumbasistemas/corubafood/model/Usuario.java]

**Produto.java**

jdk21/src/main/java/com/corumbasistemas/corubafood/model/Produto.java • 1 linhas

[ARQUIVO NAO ENCONTRADO: jdk21/src/main/java/com/corumbasistemas/corubafood/model/Produto.java]

**Mesa.java**

jdk21/src/main/java/com/corumbasistemas/corubafood/model/Mesa.java • 1 linhas

[ARQUIVO NAO ENCONTRADO: jdk21/src/main/java/com/corumbasistemas/corubafood/model/Mesa.java]

**Pedido.java**

jdk21/src/main/java/com/corumbasistemas/corubafood/model/Pedido.java • 1 linhas

[ARQUIVO NAO ENCONTRADO: jdk21/src/main/java/com/corumbasistemas/corubafood/model/Pedido.java]

4. Repositories (Acesso a Dados)

```

ProdutoRepository.java src/main/java/com/corumbasistemas/corubafood/repository/ProdutoRepository.java • 82 linhas

package com.corumbasistemas.corubafood.repository;

import com.corumbasistemas.corubafood.model.*;
import com.corumbasistemas.corubafood.util.JPAUtil;
import jakarta.persistence.*;
import java.util.List;

/**
 * ProdutoRepository - Banco NUVEM (PostgreSQL)
 * Usa empresaDocumento (CNPJ) em vez de FK
 */
public class ProdutoRepository {

    /**
     * Buscar todos os produtos de uma empresa (pelo CNPJ)
     */
    public List<Produto> buscarTodos(String empresaDocumento) {
        EntityManager em = JPAUtil.getEMNuvem();
        try {
            return em.createQuery(
                "SELECT p FROM Produto p WHERE p.empresaDocumento = :edoc AND p.ativo = true ORDER BY p.nome",
                Produto.class)
                .setParameter("edoc", empresaDocumento)
                .getResultList();
        } finally {
            em.close();
        }
    }

    /**
     * Buscar produtos por categoria
     */
    public List<Produto> buscarPorCategoria(String empresaDocumento, Long categoriaId) {
        EntityManager em = JPAUtil.getEMNuvem();
        try {
            return em.createQuery(
                "SELECT p FROM Produto p WHERE p.empresaDocumento = :edoc AND p.categoria.id = :cid AND p.ativo = true ORDER BY p.nome",
                Produto.class)
                .setParameter("edoc", empresaDocumento)
                .setParameter("cid", categoriaId)
                .getResultList();
        } finally {
            em.close();
        }
    }

    /**
     * Buscar categorias de uma empresa
     */
    public List<Categoria> buscarCategorias(String empresaDocumento) {
        EntityManager em = JPAUtil.getEMNuvem();
        try {
            return em.createQuery(
                "SELECT c FROM Categoria c WHERE c.empresaDocumento = :edoc ORDER BY c.ordem",
                Categoria.class)
                .setParameter("edoc", empresaDocumento)
                .getResultList();
        } finally {
            em.close();
        }
    }

    /**
     * Salvar produto
     */
    public Produto salvar(Produto p) {
        EntityManager em = JPAUtil.getEMNuvem();
        try {
            em.getTransaction().begin();
            if (p.getId() == null) em.persist(p);
            else p = em.merge(p);
            em.getTransaction().commit();
            return p;
        } catch (Exception e) {
            if (em.getTransaction().isActive()) em.getTransaction().rollback();
            throw e;
        } finally {
            em.close();
        }
    }
}

```

```

PagamentoRepository.java      src/main/java/com/corumbasistemas/corubafood/repository/PagamentoRepository.java • 67 linhas

package com.corumbasistemas.corubafood.repository;

import com.corumbasistemas.corubafood.model.Pagamento;
import com.corumbasistemas.corubafood.util.JPAUtil;
import jakarta.persistence.*;
import java.util.List;

/**
 * PagamentoRepository - Banco Cliente (SQLite)
 */
public class PagamentoRepository {

    public Pagamento buscarPorId(Long id) {
        EntityManager em = JPAUtil.getEMCliente();
        try {
            return em.find(Pagamento.class, id);
        } finally {
            em.close();
        }
    }

    public Pagamento buscarPorPedido(Long empresaDoc, Long pid) {
        EntityManager em = JPAUtil.getEMCliente();
        try {
            return em.createQuery(
                "SELECT p FROM Pagamento p WHERE p.empresaDocumento = :edoc AND p.pedido.id = :pid",
                Pagamento.class)
                .setParameter("edoc", empresaDoc)
                .setParameter("pid", pid)
                .getResultStream()
                .findFirst()
                .orElse(null);
        } finally {
            em.close();
        }
    }

    public List<Pagamento> buscarTodos(String empresaDocumento) {
        EntityManager em = JPAUtil.getEMCliente();
        try {
            return em.createQuery(
                "SELECT p FROM Pagamento p WHERE p.empresaDocumento = :edoc ORDER BY p.createdAt DESC",
                Pagamento.class)
                .setParameter("edoc", empresaDocumento)
                .getResultList();
        } finally {
            em.close();
        }
    }

    public Pagamento salvar(Pagamento p) {
        EntityManager em = JPAUtil.getEMCliente();
        try {
            em.getTransaction().begin();
            if (p.getId() == null) em.persist(p);
            else p = em.merge(p);
            em.getTransaction().commit();
            return p;
        } catch (Exception e) {
            if (em.getTransaction().isActive()) em.getTransaction().rollback();
            throw e;
        } finally {
            em.close();
        }
    }
}

```

```

UsuarioRepository.java        src/main/java/com/corumbasistemas/corubafood/repository/UsuarioRepository.java • 104 linhas

package com.corumbasistemas.corubafood.repository;

import com.corumbasistemas.corubafood.model.Usuario;
import com.corumbasistemas.corubafood.util.JPAUtil;
import jakarta.persistence.*;
import java.util.List;

/**
 * UsuarioRepository - Acesso a dados de usuários (SQLite local)
 * Usa empresaDocumento (String CNPJ) em vez de FK para Empresa
 */
public class UsuarioRepository {

    /**
     * Busca usuário por username e CNPJ da empresa
     */
    public Usuario buscarPorUsername(String cnpj, String username) {
        EntityManager em = JPAUtil.getEMCliente();
        try {

```



```

        return em.createQuery(
            "SELECT u FROM Usuario u WHERE u.empresaDocumento=:cnpj AND u.username=:u AND u.ativo=true",
            Usuario.class)
            .setParameter("cnpj", cnpj)
            .setParameter("u", username)
            .getSingleResult();
    } catch (NoResultException e) {
        return null;
    } finally {
        em.close();
    }
}

/**
 * Busca usuário por ID
 */
public Usuario buscarPorId(Long id) {
    EntityManager em = JPAUtil.getEMCliente();
    try {
        return em.find(Usuario.class, id);
    } finally {
        em.close();
    }
}

/**
 * Lista todos os funcionários de uma empresa (por CNPJ)
 */
public List<Usuario> buscarTodosPorCnpj(String cnpj) {
    EntityManager em = JPAUtil.getEMCliente();
    try {
        return em.createQuery(
            "SELECT u FROM Usuario u WHERE u.empresaDocumento=:cnpj AND u.ativo=true ORDER BY u.nome",
            Usuario.class)
            .setParameter("cnpj", cnpj)
            .getResultList();
    } finally {
        em.close();
    }
}

/**
 * Salva ou atualiza usuário
 */
public Usuario salvar(Usuario u) {
    EntityManager em = JPAUtil.getEMCliente();
    try {
        em.getTransaction().begin();
        if (u.getId() == null) {
            em.persist(u);
        } else {
            u = em.merge(u);
        }
        em.getTransaction().commit();
        return u;
    } catch (Exception e) {
        if (em.getTransaction().isActive()) em.getTransaction().rollback();
        throw e;
    } finally {
        em.close();
    }
}

/**
 * Exclui usuário (soft delete - desativa)
 */
public void desativar(Long id) {
    EntityManager em = JPAUtil.getEMCliente();
    try {
        em.getTransaction().begin();
        Usuario u = em.find(Usuario.class, id);
        if (u != null) {
            u.setAtivo(false);
            em.merge(u);
        }
        em.getTransaction().commit();
    } catch (Exception e) {
        if (em.getTransaction().isActive()) em.getTransaction().rollback();
        throw e;
    } finally {
        em.close();
    }
}
}

```

5. Services (Logica de Negocio)

LicencaService.java

src/main/java/com/corumbasistemas/corubafood/service/LicencaService.java • 236 linhas

```
package com.corumbasistemas.corubafood.service;

import com.corumbasistemas.corubafood.model.Empresa;
import com.corumbasistemas.corubafood.util.JPAUtil;
import jakarta.persistence.EntityManager;
import org.slf4j.*;
import java.io.*;
import java.net.*;
import java.time.*;
import java.time.format.DateTimeFormatter;

/**
 * Servico de validacao de licenca do cliente.
 *
 * FLUXO:
 * 1. Cliente ativa com codigo de ativacao (1a vez)
 * 2. Licenca fica cacheada localmente (30 dias)
 * 3. A cada login, valida com VPS se online
 * 4. Se offline, usa cache local (ate 30 dias)
 * 5. Se expirou, bloqueia o sistema
 */
public class LicencaService {
    private static final Logger logger = LoggerFactory.getLogger(LicencaService.class);
    private static final String CACHE_FILE = "licenca.cache";
    private static final String API_URL = "http://191.252.210.235:8080/api/licenca";
    private static final int DIAS_OFFLINE = 30;

    /**
     * Ativa a empresa com um codigo de ativacao (1a vez)
     */
    public boolean ativar(String codigoAtivacao) {
        try {
            Empresa empresa = buscarEmpresaLocal();
            if (empresa == null) {
                logger.error("Nenhuma empresa encontrada localmente");
                return false;
            }

            String cnpj = empresa.getDocumento();
            String json = "{\"codigo\":\"" + codigoAtivacao + "\",\"cnpj\":\"" + cnpj + "\"}";
            String response = httpPost(API_URL + "/ativar", json);

            if (response != null && response.contains("{\"status\":\"ativa\"}")) {
                salvarCache(codigoAtivacao, cnpj);
                logger.info("Licenca ativada para: " + cnpj);
                return true;
            }

            logger.warn("Ativacao falhou: " + response);
            return false;
        } catch (Exception e) {
            logger.error("Erro ao ativar: " + e.getMessage(), e);
            return false;
        }
    }

    /**
     * Valida a licenca (chamado a cada login)
     */
    public boolean validar() {
        try {
            Empresa empresa = buscarEmpresaLocal();
            if (empresa == null) {
                logger.error("Nenhuma empresa local");
                return false;
            }

            String cnpj = empresa.getDocumento();

            // Tenta validar online primeiro
            if (isOnline()) {
                String json = "{\"cnpj\":\"" + cnpj + "\"}";
                String response = httpPost(API_URL + "/validar", json);

                if (response != null) {
                    if (response.contains("{\"valida\":true}")) {
                        salvarCache("CACHED", cnpj);
                        logger.info("Licenca valida (online)");
                        return true;
                    } else {
                        logger.warn("Licenca invalida/expirada: " + response);
                        return false;
                    }
                }
            }
        }
    }
}
```

```

    }

    // Se offline, usa cache local
    return validarCacheOffline();

} catch (Exception e) {
    logger.error("Erro na validacao: " + e.getMessage());
    return validarCacheOffline();
}

}

private boolean validarCacheOffline() {
    try {
        File cache = new File(CACHE_FILE);
        if (!cache.exists()) {
            logger.warn("Sem cache local de licenca");
            return false;
        }

        BufferedReader reader = new BufferedReader(new FileReader(cache));
        String linha = reader.readLine();
        reader.close();

        if (linha == null || linha.isEmpty()) return false;

        String[] partes = linha.split("\\|");
        if (partes.length < 3) return false;

        LocalDateTime ultimaValidacao = LocalDateTime.parse(partes[2], DateTimeFormatter.ISO_LOCAL_DATE_TIME);
        long diasDesdeValidacao = Duration.between(ultimaValidacao, LocalDateTime.now()).toDays();

        if (diasDesdeValidacao > DIAS_OFFLINE) {
            logger.warn("Cache expirado (" + diasDesdeValidacao + " dias)");
            return false;
        }

        logger.info("Licenca valida (cache offline, " + diasDesdeValidacao + " dias)");
        return true;
    } catch (Exception e) {
        logger.error("Erro no cache: " + e.getMessage());
        return false;
    }
}

private void salvarCache(String codigo, String cnpj) {
    try {
        String cod = (codigo != null) ? codigo : "CACHE";
        String linha = cnpj + "|" + cod + "|" + LocalDateTime.now().format(DateTimeFormatter.ISO_LOCAL_DATE_TIME);

        FileWriter fw = new FileWriter(CACHE_FILE);
        fw.write(linha);
        fw.close();

        logger.info("Cache de licenca salvo");
    } catch (Exception e) {
        logger.error("Erro ao salvar cache: " + e.getMessage());
    }
}

private Empresa buscarEmpresaLocal() {
    EntityManager em = JPAUtil.getEMCliente();
    try {
        return em.createQuery("SELECT e FROM Empresa e WHERE e.ativo=true", Empresa.class)
            .setMaxResults(1)
            .getResultStream()
            .findFirst()
            .orElse(null);
    } catch (Exception e) {
        return null;
    } finally {
        em.close();
    }
}

private boolean isOnline() {
    try {
        URL url = new URL("http://191.252.210.235:8080/api/health");
        HttpURLConnection conn = (HttpURLConnection) url.openConnection();
        conn.setConnectTimeout(3000);
        conn.setReadTimeout(3000);
        conn.setRequestMethod("GET");
        int code = conn.getResponseCode();
        conn.disconnect();
        return code == 200;
    } catch (Exception e) {
        return false;
    }
}

private String httpPost(String urlStr, String json) {
    try {
        URL url = new URL(urlStr);
        HttpURLConnection conn = (HttpURLConnection) url.openConnection();
        conn.setRequestMethod("POST");
    }
}

```

```

        conn.setRequestProperty("Content-Type", "application/json");
        conn.setDoOutput(true);
        conn.setConnectTimeout(5000);
        conn.setReadTimeout(5000);

        OutputStream os = conn.getOutputStream();
        os.write(json.getBytes("UTF-8"));
        os.flush();
        os.close();

        int code = conn.getResponseCode();
        if (code != 200) {
            conn.disconnect();
            return null;
        }

        BufferedReader br = new BufferedReader(new InputStreamReader(conn.getInputStream(), "UTF-8"));
        StringBuilder sb = new StringBuilder();
        String line;
        while ((line = br.readLine()) != null) sb.append(line);
        br.close();
        conn.disconnect();

        return sb.toString();
    } catch (Exception e) {
        logger.debug("HTTP POST falhou: " + e.getMessage());
        return null;
    }
}

public String getInfolicenca() {
    try {
        File cache = new File(CACHE_FILE);
        if (!cache.exists()) return "Nao ativado";

        BufferedReader reader = new BufferedReader(new FileReader(cache));
        String linha = reader.readLine();
        reader.close();

        if (linha == null) return "Nao ativado";
        String[] partes = linha.split("\\|");
        if (partes.length < 3) return "Nao ativado";

        LocalDateTime ultimaValidacao = LocalDateTime.parse(partes[2], DateTimeFormatter.ISO_LOCAL_DATE_TIME);
        long dias = Duration.between(ultimaValidacao, LocalDateTime.now()).toDays();

        if (dias == 0) return "Ativado hoje";
        if (dias == 1) return "Ativado ha 1 dia";
        if (dias <= 30) return "Ativado ha " + dias + " dias";
        return "Expirado (ha " + dias + " dias sem validar)";

    } catch (Exception e) {
        return "Erro ao ler licenca";
    }
}
}

```

SyncService.java

src/main/java/com/corumbasistemas/corubafood/service/SyncService.java • 123 linhas

```

package com.corumbasistemas.corubafood.service;

import com.corumbasistemas.corubafood.util.JPAUtil;
import jakarta.persistence.EntityManager;
import org.slf4j.*;
import java.io.*;
import java.net.*;
import java.time.LocalDateTime;
import java.util.*;
import com.corumbasistemas.corubafood.model.*;

/**
 * Serviço de sincronização/backup automático.
 * Envia dados do SQLite local para o PostgreSQL na VPS.
 */
public class SyncService {
    private static final Logger logger = LoggerFactory.getLogger(SyncService.class);
    private static final String API_URL = "http://191.252.210.235:8080/api/sync";

    /**
     * Envia todos os dados da empresa para backup na VPS
     */
    public boolean sincronizarTudo() {
        try {
            if (!isOnline()) {
                logger.warn("Sem conexão - sync adiado");
                return false;
            }

            EntityManager em = JPAUtil.getEMCliente();
            Empresa empresa = em.createQuery("SELECT e FROM Empresa e WHERE e.ativo=true", Empresa.class)
                .setMaxResults(1).getResultStream().findFirst().orElse(null);

```

```

        if (empresa == null) {
            logger.warn("Nenhuma empresa para sync");
            em.close();
            return false;
        }

        String cnpj = empresa.getDocumento();
        logger.info("Iniciando sync para CNPJ: {}", cnpj);

        int total = 0;

        // Sync usuários
        List<Usuario> usuarios = em.createQuery("SELECT u FROM Usuario u WHERE u.empresa.id=:eid", Usuario.class)
            .setParameter("eid", empresa.getId()).getResultList();
        for (Usuario u : usuarios) {
            String json = "{\"tipo\":\"usuario\",\"cnpj\":\"" + cnpj + "\",\"nome\":\"" + u.getNome() + "\",\"username\":\"" +
                u.getUsername() + "\",\"apel\":\"" + u.getApel() + "\",\"ativo\":\"" + u.getAtivo() + "\"}";
            if (httpPost(API_URL, json) != null) total++;
        }

        // Sync produtos
        List<Produto> produtos = em.createQuery("SELECT p FROM Produto p WHERE p.empresa.id=:eid", Produto.class)
            .setParameter("eid", empresa.getId()).getResultList();
        for (Produto p : produtos) {
            String json = "{\"tipo\":\"produto\",\"cnpj\":\"" + cnpj + "\",\"codigo\":\"" + p.getCodigo() + "\",\"nome\":\"" +
                p.getNome() + "\",\"preco\":\"" + p.getPreco() + "\",\"ativo\":\"" + p.getAtivo() + "\"}";
            if (httpPost(API_URL, json) != null) total++;
        }

        // Sync pedidos
        List<Pedido> pedidos = em.createQuery("SELECT p FROM Pedido p WHERE p.empresa.id=:eid", Pedido.class)
            .setParameter("eid", empresa.getId()).getResultList();
        for (Pedido p : pedidos) {
            String json = "{\"tipo\":\"pedido\",\"cnpj\":\"" + cnpj + "\",\"id\":\"" + p.getId() + "\",\"total\":\"" + p.getTotal() + "\",
                \"status\":\"" + p.getStatus() + "\"}";
            if (httpPost(API_URL, json) != null) total++;
        }

        em.close();
        logger.info("Sync completo: {} registros enviados para {}", total, cnpj);
        return total > 0;
    } catch (Exception e) {
        logger.error("Erro no sync: {}", e.getMessage(), e);
        return false;
    }
}

private boolean isOnline() {
    try {
        HttpURLConnection conn = (HttpURLConnection) new URL("http://191.252.210.235:8080/api/health").openConnection();
        conn.setConnectTimeout(3000);
        conn.setReadTimeout(3000);
        conn.setRequestMethod("GET");
        int code = conn.getResponseCode();
        conn.disconnect();
        return code == 200;
    } catch (Exception e) {
        return false;
    }
}

private String httpPost(String urlStr, String json) {
    try {
        URL url = new URL(urlStr);
        HttpURLConnection conn = (HttpURLConnection) url.openConnection();
        conn.setRequestMethod("POST");
        conn.setRequestProperty("Content-Type", "application/json");
        conn.setDoOutput(true);
        conn.setConnectTimeout(10000);
        conn.setReadTimeout(30000);

        OutputStream os = conn.getOutputStream();
        os.write(json.getBytes("UTF-8"));
        os.flush();
        os.close();

        int code = conn.getResponseCode();
        BufferedReader br = new BufferedReader(new InputStreamReader(
            code == 200 ? conn.getInputStream() : conn.getErrorStream(), "UTF-8"));
        StringBuilder sb = new StringBuilder();
        String line;
        while ((line = br.readLine()) != null) sb.append(line);
        br.close();
        conn.disconnect();
        return sb.toString();
    } catch (Exception e) {
        logger.debug("HTTP falhou: {}", e.getMessage());
        return null;
    }
}
}

```

```

MesaProntaAuthService.java src/main/java/com/corumbasistemas/corubafood/service/MesaProntaAuthService.java • 331 linhas

package com.corumbasistemas.corubafood.service;

import com.corumbasistemas.corubafood.model.Empresa;
import com.corumbasistemas.corubafood.model.Usuario;
import com.corumbasistemas.corubafood.util.JPAUtil;
import jakarta.persistence.EntityManager;
import org.slf4j.*;

import java.io.*;
import java.net.*;
import java.nio.charset.StandardCharsets;

/**
 * MesaProntaAuthService - Autenticação de funcionários via API VPS
 *
 * FLUXO:
 * 1. Cliente ativa licença no MesaPronta (código + CNPJ)
 * 2. FastAPI cria funcionário ADMIN no PostgreSQL (VPS)
 * 3. Dono do restaurante cadastra funcionários no Portal do Cliente (web)
 * 4. MesaPronta autentica funcionários consultando API VPS
 * 5. Sincroniza funcionários do VPS para SQLite local
 */
public class MesaProntaAuthService {
    private static final Logger logger = LoggerFactory.getLogger(MesaProntaAuthService.class);
    private static final String API_URL = "http://191.252.210.235:8080/api/mesapronta";

    /**
     * Resultado da autenticação
     */
    public static class AuthResult {
        public boolean sucesso;
        public String erro;
        public int funcionarioId;
        public String nome;
        public String username;
        public String papel;
        public int empresaId;

        public static AuthResult erro(String msg) {
            AuthResult r = new AuthResult();
            r.sucesso = false;
            r.erro = msg;
            return r;
        }

        public static AuthResult ok(int fid, String nome, String username, String papel, int eid) {
            AuthResult r = new AuthResult();
            r.sucesso = true;
            r.funcionarioId = fid;
            r.nome = nome;
            r.username = username;
            r.papel = papel;
            r.empresaId = eid;
            return r;
        }
    }

    /**
     * Autentica funcionário contra API VPS
     */
    public AuthResult autenticar(String username, String senha, String cnpj) {
        try {
            // Buscar CNPJ se não fornecido
            if (cnpj == null || cnpj.isEmpty()) {
                cnpj = buscarCnpjLocal();
                if (cnpj == null) {
                    return AuthResult.erro("CNPJ não encontrado. Ative a licença primeiro.");
                }
            }

            String json = "{\"username\":\"" + escape(username) + "\",\" "
                + "\"senha\":\"" + escape(senha) + "\",\" "
                + "\"cnpj\":\"" + escape(cnpj) + "\"}";

            String response = httpPost(API_URL + "/auth", json);
            logger.debug("Auth response: " + response);

            if (response == null) {
                return AuthResult.erro("Erro de conexão com servidor");
            }

            if (response.contains("{\"sucesso\":true}") {
                // Parse manual (sem biblioteca JSON)
                int fid = getIntField(response, "id");
                String nome = getStringField(response, "nome");
                String user = getStringField(response, "username");
                String papel = getStringField(response, "papel");
                int eid = getIntField(response, "empresa_id");

                logger.info("Auth VPS OK: {} ({}), username, papel);
                return AuthResult.ok(fid, nome, user, papel, eid);
            } else {

```

```

        String erro = getStringField(response, "erro");
        if (erro == null) erro = "Falha na autenticação";
        logger.warn("Auth VPS falhou: {} - {}", username, erro);
        return AuthResult.erro(erro);
    }
} catch (Exception e) {
    logger.error("Erro auth VPS: {}", e.getMessage(), e);
    return AuthResult.erro("Erro interno: " + e.getMessage());
}
}

/**
 * Baixa funcionários do VPS e salva no SQLite local
 * Chamado após login bem-sucedido para manter sync
 */
public int sincronizarFuncionarios(String cnpj) {
    try {
        if (cnpj == null || cnpj.isEmpty()) {
            cnpj = buscarCnpjLocal();
            if (cnpj == null) return 0;
        }

        String response = httpGet(API_URL + "/funcionarios/" + cnpj);
        if (response == null || response.contains("\\"erro\\"")) {
            logger.warn("Sync funcionários falhou: {}", response);
            return 0;
        }

        // Parse dos funcionários (parse manual simples)
        int count = 0;
        EntityManager em = JPAUtil.getEMCliente();
        try {
            em.getTransaction().begin();

            // Extrair array de funcionários
            int arrStart = response.indexOf("[");
            int arrEnd = response.lastIndexOf("]");
            if (arrStart < 0 || arrEnd < 0) return 0;

            String arr = response.substring(arrStart + 1, arrEnd);
            // Split por objetos {}
            int depth = 0;
            int objStart = -1;
            for (int i = 0; i < arr.length(); i++) {
                char c = arr.charAt(i);
                if (c == '{') {
                    if (depth == 0) objStart = i;
                    depth++;
                } else if (c == '}') {
                    depth--;
                    if (depth == 0 && objStart >= 0) {
                        String obj = arr.substring(objStart + 1, i);
                        salvarFuncionarioLocal(em, obj, cnpj);
                        count++;
                        objStart = -1;
                    }
                }
            }

            em.getTransaction().commit();
            logger.info("Sync OK: {} funcionários baixados", count);
            return count;
        } catch (Exception e) {
            if (em.getTransaction().isActive()) em.getTransaction().rollback();
            logger.error("Erro ao salvar funcionários: {}", e.getMessage());
            return 0;
        } finally {
            em.close();
        }
    } catch (Exception e) {
        logger.error("Erro sync funcionários: {}", e.getMessage());
        return 0;
    }
}

/**
 * Salva ou atualiza funcionário no SQLite local
 */
@SuppressWarnings("unchecked")
private void salvarFuncionarioLocal(EntityManager em, String obj, String cnpj) {
    try {
        int id = getIntField(obj, "id");
        String nome = getStringField(obj, "nome");
        String username = getStringField(obj, "username");
        String senha = getStringField(obj, "senha");
        String papelStr = getStringField(obj, "papel");
        boolean ativo = obj.contains("\\"ativo\\":true");

        if (nome == null || username == null) return;

        // Buscar empresa local
        Empresa empresa = em.createQuery(
            "SELECT e FROM Empresa e WHERE e.documento=:d", Empresa.class)
            .setParameter("d", cnpj)
            .getResultStream().findFirst().orElse(null);
    }
}

```

```

        if (empresa == null) {
            logger.warn("Empresa local não encontrada para CNPJ: {}", cnpj);
            return;
        }

        // Buscar funcionário existente por username
        Usuario existente = null;
        try {
            existente = em.createQuery(
                "SELECT u FROM Usuario u WHERE u.username=:u AND u.empresaDocumento=:c",
                Usuario.class)
                .setParameter("u", username)
                .setParameter("c", cnpj)
                .getSingleResult();
        } catch (Exception e) {
            // Não encontrado = novo
        }

        Usuario.Papel papel;
        try {
            papel = Usuario.Papel.valueOf(papelStr);
        } catch (Exception e) {
            papel = Usuario.Papel.GARCOM;
        }

        if (existente != null) {
            existente.setNome(nome);
            existente.setSenha(senha != null ? senha : existente.getSenha());
            existente.setPapel(papel);
            existente.setAtivo(ativo);
            em.merge(existente);
        } else {
            Usuario novo = new Usuario(nome, username, senha, papel, cnpj);
            em.persist(novo);
        }
    } catch (Exception e) {
        logger.warn("Erro ao processar funcionário: {}", e.getMessage());
    }
}

/**
 * Busca CNPJ da empresa local (SQLite)
 */
private String buscarCnpjLocal() {
    EntityManager em = JPAUtil.getEMCliente();
    try {
        Empresa e = em.createQuery(
            "SELECT e FROM Empresa e WHERE e.ativo=true", Empresa.class)
            .setMaxResults(1).getResultStream().findFirst().orElse(null);
        return e != null ? e.getDocumento() : null;
    } catch (Exception ex) {
        return null;
    } finally {
        em.close();
    }
}

// ===== Utilidades HTTP =====

private String httpPost(String urlStr, String json) {
    try {
        URL url = new URL(urlStr);
        HttpURLConnection conn = (HttpURLConnection) url.openConnection();
        conn.setRequestMethod("POST");
        conn.setRequestProperty("Content-Type", "application/json");
        conn.setDoOutput(true);
        conn.setConnectTimeout(5000);
        conn.setReadTimeout(5000);

        try (OutputStream os = conn.getOutputStream()) {
            os.write(json.getBytes(StandardCharsets.UTF_8));
        }

        return readResponse(conn);
    } catch (Exception e) {
        logger.debug("POST falhou: {}", e.getMessage());
        return null;
    }
}

private String httpGet(String urlStr) {
    try {
        URL url = new URL(urlStr);
        HttpURLConnection conn = (HttpURLConnection) url.openConnection();
        conn.setRequestMethod("GET");
        conn.setConnectTimeout(5000);
        conn.setReadTimeout(5000);

        return readResponse(conn);
    } catch (Exception e) {
        logger.debug("GET falhou: {}", e.getMessage());
        return null;
    }
}

```



```

private String readResponse(HttpURLConnection conn) throws IOException {
    int code = conn.getResponseCode();
    InputStream is = (code >= 200 && code < 300) ? conn.getInputStream() : conn.getErrorStream();
    if (is == null) return null;

    BufferedReader br = new BufferedReader(new InputStreamReader(is, StandardCharsets.UTF_8));
    StringBuilder sb = new StringBuilder();
    String line;
    while ((line = br.readLine()) != null) sb.append(line);
    br.close();
    conn.disconnect();
    return sb.toString();
}

// ===== Parse JSON simples =====

private String getStringField(String json, String field) {
    String key = "\"" + field + "\"";
    int start = json.indexOf(key);
    if (start < 0) return null;
    start += key.length();
    int end = json.indexOf(":", start);
    if (end < 0) return null;
    return json.substring(start, end).replace("\\\"", "");
}

private int getIntField(String json, String field) {
    String key = "\"" + field + "\"";
    int start = json.indexOf(key);
    if (start < 0) return 0;
    start += key.length();
    // Pular aspas se for string
    if (start < json.length() && json.charAt(start) == '"') start++;
    int end = start;
    while (end < json.length() && (Character.isDigit(json.charAt(end)) || json.charAt(end) == '-')) end++;
    try {
        return Integer.parseInt(json.substring(start, end));
    } catch (NumberFormatException e) {
        return 0;
    }
}

private String escape(String s) {
    if (s == null) return "";
    return s.replace("\\", "\\\\").replace("\"", "\\\"").replace("\n", "\\n");
}
}

```

6. Controllers (Logica de Interface)

```

LoginController.java src/main/java/com/corumbasistemas/corubafood/controller/LoginController.java • 209 linhas

package com.corumbasistemas.corubafood.controller;

import com.corumbasistemas.corubafood.CorubaFoodApp;
import com.corumbasistemas.corubafood.model.*;
import com.corumbasistemas.corubafood.service.LicencaService;
import com.corumbasistemas.corubafood.service.MesaProntaAuthService;
import com.corumbasistemas.corubafood.service.MesaProntaAuthService.AuthResult;
import com.corumbasistemas.corubafood.repository.UsuarioRepository;
import javafx.application.Platform;
import javafx.fxml.FXML;
import javafx.scene.control.*;
import javafx.scene.paint.Color;
import org.slf4j.*;

/**
 * LoginController - Login do MesaPronta
 *
 * FLUXO:
 * 1. Funcionário digita username + senha (sem CNPJ - sistema resolve automaticamente)
 * 2. Sistema valida licença (online ou cache offline)
 * 3. Sistema autentica contra API VPS (funcionários cadastrados no Portal do Cliente)
 * 4. Se VPS offline, usa SQLite local (fallback)
 * 5. Sincroniza funcionários do VPS para local
 * 6. Login OK → mostra tela principal com permissões do funcionário
 */
public class LoginController {
    private static final Logger logger = LoggerFactory.getLogger(LoginController.class);
    private final LicencaService licencaService = new LicencaService();
    private final MesaProntaAuthService mpAuth = new MesaProntaAuthService();
    private final UsuarioRepository usuarioRepo = new UsuarioRepository();

    @FXML private TextField txtUsername;
    @FXML private PasswordField txtSenha;
    @FXML private Label lblMensagem;
    @FXML private Label lblLicenca;
    @FXML private Button btnEntrar;
    @FXML private ProgressIndicator progress;
    @FXML private Hyperlink linkAtivacao;
    @FXML private Hyperlink linkRestauracao;

    @FXML
    public void initialize() {
        lblMensagem.setText("");
        progress.setVisible(false);

        // Mostrar info da licença
        String info = licencaService.getInfoLicenca();
        if (info.equals("Não ativado") || info.startsWith("Expirado")) {
            lblLicenca.setTextFill(Color.ORANGE);
            linkAtivacao.setVisible(true);
        } else {
            lblLicenca.setTextFill(Color.LIGHTGREEN);
            linkAtivacao.setVisible(false);
        }
        lblLicenca.setText("Licença: " + info);
    }

    /**
     * Tenta fazer login de um funcionário.
     * 1. Valida licença local primeiro (rápido)
     * 2. Tenta autenticar via API VPS (funcionários do Portal do Cliente)
     * 3. Se VPS offline, usa SQLite local como fallback
     */
    @FXML
    private void handleEntrar() {
        String user = txtUsername.getText().trim();
        String senha = txtSenha.getText();

        if (user.isEmpty() || senha.isEmpty()) {
            lblMensagem.setTextFill(Color.RED);
            lblMensagem.setText("Preencha usuário e senha!");
            return;
        }

        progress.setVisible(true);
        btnEntrar.setDisable(true);
        lblMensagem.setTextFill(Color.WHITE);
        lblMensagem.setText("Verificando...");

        new Thread(() -> {
            // 1. Validar licença (local + online)
            boolean licencaValida = licencaService.validar();
            if (!licencaValida) {
                Platform.runLater(() -> {

```

```

        progress.setVisible(false);
        btnEntrar.setDisable(false);
        lblMensagem.setTextFill(Color.RED);
        lblMensagem.setText("❌ Licença inválida ou expirada.\nClique em \"Ativar\" ou contate o suporte.");
    });
    return;
}

// 2. Buscar empresa local (para obter CNPJ automaticamente)
Empresa empresa = CorubaFoodApp.getAuthController().buscarEmpresaLocal();
if (empresa == null) {
    Platform.runLater(() -> {
        progress.setVisible(false);
        btnEntrar.setDisable(false);
        lblMensagem.setTextFill(Color.RED);
        lblMensagem.setText("❌ Empresa não encontrada.\nAtive sua licença primeiro.");
    });
    return;
}

String cnpj = empresa.getDocumento();

// 3. Tentar autenticar via API VPS (funcionários do Portal do Cliente)
AuthResult auth = mpAuth.autenticar(user, senha, cnpj);

if (!auth.sucesso) {
    // FALLBACK: tentar autenticar localmente (SQLite)
    logger.warn("Auth VPS falhou ({}), tentando SQLite local", auth.erro);
    Usuario local = CorubaFoodApp.getAuthController().autenticar(cnpj, user, senha);

    if (local == null) {
        Platform.runLater(() -> {
            progress.setVisible(false);
            btnEntrar.setDisable(false);
            lblMensagem.setTextFill(Color.RED);
            lblMensagem.setText("❌ Usuário ou senha inválidos!");
        });
        return;
    }

    // Login OK via SQLite local
    final Usuario usuarioLocal = local;
    // Tentar sync em background
    new Thread(() -> mpAuth.sincronizarFuncionarios(cnpj)).start();

    Platform.runLater(() -> {
        CorubaFoodApp.setUsuarioLogado(usuarioLocal);
        logger.info("Login OK (local): {} ({})", user, usuarioLocal.getPapel());
        mostrarTelaPrincipal();
    });
    return;
}

// 4. Auth VPS OK! Buscar ou criar usuário no SQLite local
Usuario usuarioLocal = CorubaFoodApp.getAuthController()
    .autenticar(cnpj, user, senha);

if (usuarioLocal == null) {
    // Funcionário existe no VPS mas não no local - criar local
    logger.info("Criando funcionário local: {}", user);
    Usuario.Papel papel;
    try {
        papel = Usuario.Papel.valueOf(auth.papel);
    } catch (Exception e) {
        papel = Usuario.Papel.GARCOM;
    }
    usuarioLocal = new Usuario(auth.nome, user, senha, papel, cnpj);
    usuarioLocal = usuarioRepo.salvar(usuarioLocal);
}

// 5. Sync funcionários em background
final Usuario finalUsuario = usuarioLocal;
new Thread(() -> {
    mpAuth.sincronizarFuncionarios(cnpj);
}).start();

Platform.runLater(() -> {
    progress.setVisible(false);
    btnEntrar.setDisable(false);
    CorubaFoodApp.setUsuarioLogado(finalUsuario);
    logger.info("Login OK (VPS): {} ({})", user, auth.papel);
    mostrarTelaPrincipal();
});
}).start();
}

private void mostrarTelaPrincipal() {
    try {
        CorubaFoodApp.mostrarPrincipal();
    } catch (Exception ex) {
        logger.error("Erro ao mostrar principal: {}", ex.getMessage(), ex);
        lblMensagem.setTextFill(Color.RED);
        lblMensagem.setText("Erro ao carregar sistema!");
    }
}
}

```

```

@FXML
private void handleAtivacao() {
    try {
        javafx.fxml.FXMLLoader l = new javafx.fxml.FXMLLoader(
            getClass().getResource("/view/AtivacaoView.fxml"));
        javafx.scene.Parent r = l.load();
        CorubaFoodApp.getPS().getScene().setRoot(r);
        CorubaFoodApp.getPS().setTitle("Corumbá Food - Ativação");
    } catch (Exception e) {
        logger.error("Erro ao abrir ativação: {}", e.getMessage());
    }
}

@FXML
private void handleRestauracao() {
    try {
        javafx.fxml.FXMLLoader l = new javafx.fxml.FXMLLoader(
            getClass().getResource("/view/RestauracaoView.fxml"));
        javafx.scene.Parent r = l.load();
        CorubaFoodApp.getPS().getScene().setRoot(r);
        CorubaFoodApp.getPS().setTitle("Corumbá Food - Restauração");
    } catch (Exception e) {
        logger.error("Erro ao abrir restauração: {}", e.getMessage());
        lblMensagem.setTextFill(Color.RED);
        lblMensagem.setText("Tela de restauração não disponível.");
    }
}
}

```

AuthController.java

src/main/java/com/corumbasistemas/corubafood/controller/AuthController.java • 87 linhas

```

package com.corumbasistemas.corubafood.controller;

import com.corumbasistemas.corubafood.model.*;
import com.corumbasistemas.corubafood.repository.UsuarioRepository;
import com.corumbasistemas.corubafood.security.PasswordHash;
import com.corumbasistemas.corubafood.util.JPAUtil;
import jakarta.persistence.*;
import org.slf4j.*;

/**
 * AuthController - Autenticação de funcionários
 *
 * Usa empresaDocumento (String CNPJ) para identificar a empresa,
 * compatível com o banco SQLite local.
 */
public class AuthController {
    private static final Logger logger = LoggerFactory.getLogger(AuthController.class);
    private final UsuarioRepository repo = new UsuarioRepository();

    /**
     * Autentica usuário pelo username + CNPJ da empresa
     */
    public Usuario autenticar(String cnpj, String username, String senha) {
        try {
            Usuario usr = repo.buscarPorUsername(cnpj, username);
            if (usr == null) {
                PasswordHash.hash("dummy"); // Timing-safe
                logger.warn("Login invalido: {} / {}", username, cnpj);
                return null;
            }
            if (!PasswordHash.verify(senha, usr.getSenha())) {
                logger.warn("Senha errada: {}", username);
                return null;
            }
            if (PasswordHash.needsRehash(usr.getSenha())) {
                logger.info("Migrando hash da senha: {}", username);
                usr.setSenha(PasswordHash.hash(senha));
                repo.salvar(usr);
            }
            logger.info("Login OK: {} ({}), username, cnpj);
            return usr;
        } catch (Exception e) {
            logger.error("Erro auth: {}", e.getMessage(), e);
            return null;
        }
    }

    /**
     * Busca empresa pelo documento (CNPJ) no SQLite local
     */
    public Empresa buscarEmpresaPorDocumento(String doc) {
        EntityManager em = JPAUtil.getEMCliente();
        try {
            return em.createQuery("SELECT e FROM Empresa e WHERE e.documento=:d AND e.ativo=true", Empresa.class)
                .setParameter("d", doc)
                .getSingleResult();
        } catch (NoResultException e) {
            return null;
        } finally {
            em.close();
        }
    }
}

```

```

    }
}

/**
 * Busca a única empresa local (para login sem CNPJ)
 */
public Empresa buscarEmpresaLocal() {
    EntityManager em = JPAUtil.getEMCliente();
    try {
        return em.createQuery("SELECT e FROM Empresa e WHERE e.ativo=true", Empresa.class)
            .setMaxResults(1)
            .getResultStream()
            .findFirst()
            .orElse(null);
    } catch (Exception e) {
        logger.error("Erro ao buscar empresa local: {}", e.getMessage());
        return null;
    } finally {
        em.close();
    }
}

public boolean isAdmin(Usuario u) {
    return u != null && u.getPapel() == Usuario.Papel.ADMIN;
}
}

```

AtivacaoController.java src/main/java/com/corumbasistemas/corubafood/controller/AtivacaoController.java • 70 linhas

```

package com.corumbasistemas.corubafood.controller;

import com.corumbasistemas.corubafood.CorubaFoodApp;
import com.corumbasistemas.corubafood.service.LicencaService;
import javafx.application.Platform;
import javafx.fxml.FXML;
import javafx.scene.control.*;
import javafx.scene.paint.Color;
import org.slf4j.*;

public class AtivacaoController {
    private static final Logger logger = LoggerFactory.getLogger(AtivacaoController.class);
    private final LicencaService licencaService = new LicencaService();

    @FXML private TextField txtCodigoAtivacao;
    @FXML private Label lblMensagem;
    @FXML private Button btnAtivar;
    @FXML private ProgressIndicator progress;

    @FXML
    private void handleAtivar() {
        String codigo = txtCodigoAtivacao.getText().trim();
        if (codigo.isEmpty()) {
            lblMensagem.setTextFill(Color.RED);
            lblMensagem.setText("Digite o código de ativação!");
            return;
        }

        progress.setVisible(true);
        btnAtivar.setDisable(true);
        lblMensagem.setTextFill(Color.WHITE);
        lblMensagem.setText("Ativando...");

        // Rodar em thread separada para não travar a UI
        new Thread(() -> {
            boolean sucesso = licencaService.ativar(codigo);

            Platform.runLater(() -> {
                progress.setVisible(false);
                btnAtivar.setDisable(false);

                if (sucesso) {
                    lblMensagem.setTextFill(Color.LIGHTGREEN);
                    lblMensagem.setText("✅ Licença ativada com sucesso!\nFeche e abra o app para entrar.");

                    // Voltar ao login após 3 segundos
                    new Thread(() -> {
                        try { Thread.sleep(3000); } catch (InterruptedException e) {}
                        Platform.runLater(() -> {
                            try { CorubaFoodApp.mostrarLogin(); } catch (Exception e) {}
                        });
                    }).start();
                } else {
                    lblMensagem.setTextFill(Color.RED);
                    lblMensagem.setText("❌ Código inválido ou sem conexão.\nVerifique o código e tente novamente.");
                }
            });
        }).start();
    }

    @FXML
    private void handleVoltar() {
        try {

```

```

        CorubaFoodApp.mostrarLogin();
    } catch (Exception e) {
        logger.error("Erro ao voltar: {}", e.getMessage());
    }
}
}

```

 **PrincipalController.java** src/main/java/com/corumbasistemas/corubafood/controller/PrincipalController.java • 201 linhas

```

package com.corumbasistemas.corubafood.controller;

import com.corumbasistemas.corubafood.CorubaFoodApp;
import com.corumbasistemas.corubafood.model.*;
import com.corumbasistemas.corubafood.repository.ProdutoRepository;
import com.corumbasistemas.corubafood.repository.UsuarioRepository;
import com.corumbasistemas.corubafood.util.JPAUtil;
import javafx.collections.FXCollections;
import javafx.collections.ObservableList;
import javafx.fxml.FXML;
import javafx.fxml.Initializable;
import javafx.geometry.Insets;
import javafx.geometry.Pos;
import javafx.scene.control.*;
import javafx.scene.layout.*;
import javafx.scene.paint.Color;
import javafx.scene.text.Font;
import javafx.scene.text.FontWeight;
import org.slf4j.*;

import java.math.BigDecimal;
import java.net.URL;
import java.util.ResourceBundle;

public class PrincipalController implements Initializable {
    private static final Logger logger = LoggerFactory.getLogger(PrincipalController.class);

    @FXML private Label lblUsuario;
    @FXML private Label lblMesaTitulo;
    @FXML private Label lblMesaStatus;
    @FXML private Label lblTotal;
    @FXML private Label lblStatusBar;
    @FXML private ListView<String> lvItens;
    @FXML private FlowPane fpMesas;

    private final ProdutoRepository produtoRepo = new ProdutoRepository();
    private Mesa mesaSelecionada;
    private Pedido pedidoAtual;
    private final ObservableList<String> itensLista = FXCollections.observableArrayList();

    @Override
    public void initialize(URL url, ResourceBundle rb) {
        Usuario u = CorubaFoodApp.getUsuarioLogado();
        if (u != null) {
            lblUsuario.setText(u.getNome() + " (" + u.getPapel() + ")");
        }
        lvItens.setItems(itensLista);
        carregarMesas();
        lblStatusBar.setText("Selecione uma mesa para começar");
    }

    private void carregarMesas() {
        fpMesas.getChildren().clear();
        Usuario u = CorubaFoodApp.getUsuarioLogado();
        if (u == null) return;

        try {
            var em = JPAUtil.getEMCliente();
            var mesas = em.createQuery(
                "SELECT m FROM Mesa m WHERE m.empresaDocumento=:eid ORDER BY m.numero", Mesa.class)
                .setParameter("edoc", u.getEmpresaDocumento())
                .getResultList();
            em.close();

            for (Mesa m : mesas) {
                Button btn = new Button("Mesa " + m.getNumero() + "\n" + m.getCapacidade() + " lugares");
                btn.setPrefSize(120, 80);
                btn.setFont(Font.font(12));
                btn.setAlignment(Pos.CENTER);

                switch (m.getStatus()) {
                    case LIVRE -> {
                        btn.setStyle("-fx-background-color: #4eacca3; -fx-text-fill: #0f0f23; -fx-font-weight: bold;");
                        btn.setOnAction(e -> selecionarMesa(m, btn));
                    }
                    case OCUPADA -> {
                        btn.setStyle("-fx-background-color: #e94560; -fx-text-fill: white; -fx-font-weight: bold;");
                        btn.setOnAction(e -> selecionarMesa(m, btn));
                    }
                    case RESERVADA -> {
                        btn.setStyle("-fx-background-color: #f77f00; -fx-text-fill: white; -fx-font-weight: bold;");
                        btn.setOnAction(e -> selecionarMesa(m, btn));
                    }
                }
            }
        }
    }
}

```

```

    }
    fpMesas.getChildren().add(btn);
}
lblStatusBar.setText(mesas.size() + " mesas carregadas");
} catch (Exception e) {
    logger.error("Erro ao carregar mesas: {}", e.getMessage());
    lblStatusBar.setText("Erro ao carregar mesas");
}
}

private void selecionarMesa(Mesa mesa, Button btn) {
    this.mesaSelecionada = mesa;
    lblMesaTitulo.setText("Mesa " + mesa.getNumero() + " (" + mesa.getCapacidade() + " lugares)");
    lblMesaStatus.setText(mesa.getStatus().name());

    // Criar novo pedido se mesa livre
    if (mesa.getStatus() == Mesa.Status.LIVRE) {
        pedidoAtual = new Pedido();
        pedidoAtual.setMesaId(mesa.getId());
        pedidoAtual.setUsuarioId(CorubaFoodApp.getUsuarioLogado().getId());
        // Empresa vem do CNPJ do usuario logado
        pedidoAtual.setStatus(Pedido.Status.ABERTO);
        itensLista.clear();
        lblTotal.setText("R$ 0,00");
        lblStatusBar.setText("Mesa " + mesa.getNumero() + " - Novo pedido");
    } else {
        lblStatusBar.setText("Mesa " + mesa.getNumero() + " - Ocupada");
    }
}

@FXML
private void handleAdicionarItem() {
    if (mesaSelecionada == null) {
        lblStatusBar.setText("⚠ Seleccione uma mesa primeiro!");
        return;
    }
    Usuario u = CorubaFoodApp.getUsuarioLogado();
    if (u == null) return;

    try {
        var produtos = produtoRepo.buscarTodos(u.getEmpresaDocumento());
        if (produtos.isEmpty()) {
            lblStatusBar.setText("⚠ Nenhum produto cadastrado");
            return;
        }

        // Pega um produto aleatório para simular
        var prod = produtos.get((int)(Math.random() * produtos.size()));
        int qtd = (int)(Math.random() * 3) + 1;

        ItemPedido item = new ItemPedido();
        item.setProduto(prod);
        item.setQuantidade(qtd);
        item.setPrecoUnitario(prod.getPreco());
        item.setSubtotal(item.getPrecoUnitario().multiply(BigDecimal.valueOf(qtd)));

        itensLista.add(qtd + "x " + prod.getNome() + " - R$ " + item.getSubtotal());

        // Atualizar total
        BigDecimal totalAtual = BigDecimal.ZERO;
        for (String s : itensLista) {
            // parse simples
        }
        lblTotal.setText("R$ " + item.getSubtotal().multiply(BigDecimal.valueOf(qtd)));
        lblStatusBar.setText("✅ " + qtd + "x " + prod.getNome() + " adicionado");
    } catch (Exception e) {
        logger.error("Erro ao adicionar item: {}", e.getMessage());
        lblStatusBar.setText("❌ Erro ao adicionar item");
    }
}

@FXML
private void handleRemoverItem() {
    int idx = lvItens.getSelectionModel().getSelectedIndex();
    if (idx >= 0) {
        itensLista.remove(idx);
        lblStatusBar.setText("Item removido");
    }
}

@FXML
private void handleFecharPedido() {
    if (mesaSelecionada == null || itensLista.isEmpty()) {
        lblStatusBar.setText("⚠ Nenhum item no pedido");
        return;
    }
    lblStatusBar.setText("✅ Pedido fechado - Total: " + lblTotal.getText());
}

@FXML
private void handlePagamento() {
    if (mesaSelecionada == null) {
        lblStatusBar.setText("⚠ Seleccione uma mesa");
        return;
    }
}

```

```

        lblStatusBar.setText("💰 Pagamento registrado - " + lblTotal.getText());
    }

    @FXML
    private void handleSair() {
        try {
            CorubaFoodApp.mostrarLogin();
        } catch (Exception e) {
            logger.error("Erro ao sair: {}", e.getMessage());
        }
    }

    @FXML
    private void handleDelivery() {
        try {
            CorubaFoodApp.mostrarDelivery();
        } catch (Exception e) {
            logger.error("Erro ao abrir delivery: {}", e.getMessage());
            lblStatusBar.setText("❌ Erro ao abrir Delivery: " + e.getMessage());
        }
    }
}

```

 **PagamentoController.java** src/main/java/com/corumbasistemas/corubafood/controller/PagamentoController.java • 44 linhas

```

package com.corumbasistemas.corubafood.controller;

import com.corumbasistemas.corubafood.model.*;
import com.corumbasistemas.corubafood.repository.PagamentoRepository;
import org.slf4j.*;
import java.math.BigDecimal;

/**
 * PagamentoController - Controle de pagamentos e descontos
 */
public class PagamentoController {
    private static final Logger logger = LoggerFactory.getLogger(PagamentoController.class);
    private final PagamentoRepository repo = new PagamentoRepository();
    private final AuthController auth;

    public PagamentoController(AuthController auth) { this.auth = auth; }

    /**
     * Aplicar desconto em pedido (apenas admin)
     * Usa a nova API de autenticação (username + senha)
     */
    public boolean aplicarDesconto(Long pid, BigDecimal desc, String user, String senha) {
        // Nova API: autenticar pelo username (CNPJ vem do BD)
        Usuario a = auth.autenticar(user, senha, null);
        if (a == null || !auth.isAdmin(a)) {
            logger.warn("Desconto não autorizado: {}", user);
            return false;
        }
        try {
            var p = repo.buscarPorId(pid);
            if (p != null) {
                p.setDesconto(desc);
                repo.salvar(p);
                logger.info("Desconto {} no pedido {} por {}", desc, pid, user);
                return true;
            }
            return false;
        } catch (Exception e) {
            logger.error("Erro desconto: {}", e.getMessage(), e);
            return false;
        }
    }
}

```

 **ConfigIntegracaoController.java** src/main/java/com/corumbasistemas/corubafood/controller/ConfigIntegracaoController.java • 267 linhas

```

package com.corumbasistemas.corubafood.controller;

import javafx.fxml.FXML;
import javafx.fxml.Initializable;
import javafx.scene.control.*;
import javafx.scene.layout.VBox;
import javafx.scene.paint.Color;
import javafx.scene.text.Font;
import javafx.scene.text.FontWeight;
import org.slf4j.*;

import java.io.*;
import java.net.URL;
import java.util.Properties;
import java.util.ResourceBundle;

```



```

/**
 * ConfigIntegracaoController - Tela de configuração das APIs.
 *
 * O cliente cola as credenciais da iFood e 99Food aqui.
 * As credenciais são salvas em um arquivo .properties local.
 *
 * Como obter as credenciais:
 *
 * iFood:
 * 1. Acesse developer.ifood.com.br
 * 2. Registre-se como parceiro merchant
 * 3. Crie um app para obter client_id e client_secret
 * 4. Copie o merchant_id do seu estabelecimento
 *
 * 99Food:
 * 1. Acesse o painel do parceiro 99Food
 * 2. Solicite acesso à API pelo suporte
 * 3. Receba client_id, client_secret e merchant_id
 */
public class ConfigIntegracaoController implements Initializable {
    private static final Logger logger = LoggerFactory.getLogger(ConfigIntegracaoController.class);
    private static final String CONFIG_FILE = "integrations.properties";

    // ===== iFood =====
    @FXML private TextField txtIfoodClientId;
    @FXML private PasswordField txtIfoodClientSecret;
    @FXML private TextField txtIfoodMerchantId;
    @FXML private Label lblIfoodStatus;
    @FXML private Button btnIfoodTestar;
    @FXML private Button btnIfoodSalvar;

    // ===== 99Food =====
    @FXML private TextField txt99ClientId;
    @FXML private PasswordField txt99ClientSecret;
    @FXML private TextField txt99MerchantId;
    @FXML private Label lbl99Status;
    @FXML private Button btn99Testar;
    @FXML private Button btn99Salvar;

    // ===== Geral =====
    @FXML private Label lblStatusBar;
    @FXML private Spinner<Integer> spnPollInterval;
    @FXML private CheckBox chkAutoStart;
    @FXML private CheckBox chkSoundAlert;

    private Properties config;

    @Override
    public void initialize(URL url, ResourceBundle rb) {
        config = new Properties();
        loadConfig();

        // Configurar spinner de intervalo (10-300 segundos)
        SpinnerValueFactory<Integer> factory = new SpinnerValueFactory.IntegerSpinnerValueFactory(
            10, 300, Integer.parseInt(config.getProperty("sync.poll_interval", "30"))
        );
        spnPollInterval.setValueFactory(factory);

        // Preencher campos iFood
        txtIfoodClientId.setText(config.getProperty("ifood.client_id", ""));
        txtIfoodClientSecret.setText(config.getProperty("ifood.client_secret", ""));
        txtIfoodMerchantId.setText(config.getProperty("ifood.merchant_id", ""));

        // Preencher campos 99Food
        txt99ClientId.setText(config.getProperty("food99.client_id", ""));
        txt99ClientSecret.setText(config.getProperty("food99.client_secret", ""));
        txt99MerchantId.setText(config.getProperty("food99.merchant_id", ""));

        // Checkboxes
        chkAutoStart.setSelected("true".equals(config.getProperty("sync.auto_start", "true")));
        chkSoundAlert.setSelected("true".equals(config.getProperty("sync.sound_alert", "true")));

        updateStatusLabels();
        lblStatusBar.setText("Configuração carregada. Preencha as credenciais e clique em Testar.");
    }

    /**
     * Salva configurações do iFood
     */
    @FXML
    private void handleIfoodSalvar() {
        config.setProperty("ifood.client_id", txtIfoodClientId.getText().trim());
        config.setProperty("ifood.client_secret", txtIfoodClientSecret.getText().trim());
        config.setProperty("ifood.merchant_id", txtIfoodMerchantId.getText().trim());
        saveConfig();
        lblStatusBar.setText("✅ Credenciais iFood salvas!");
        logger.info("Config iFood salva (merchant: {})", txtIfoodMerchantId.getText().trim());
    }

    /**
     * Configurações do 99Food
     */
    @FXML
    private void handle99Salvar() {

```

```

config.setProperty("food99.client_id", txt99ClientId.getText().trim());
config.setProperty("food99.client_secret", txt99ClientSecret.getText().trim());
config.setProperty("food99.merchant_id", txt99MerchantId.getText().trim());
saveConfig();
lblStatusBar.setText("✅ Credenciais 99Food salvas!");
logger.info("Config 99Food salva (merchant: {})", txt99MerchantId.getText().trim());
}

/**
 * Testa conexão com iFood
 * (Em produção, faz uma chamada real de API)
 */
@FXML
private void handleIfoodTestar() {
    String clientId = txtIfoodClientId.getText().trim();
    String clientSecret = txtIfoodClientSecret.getText().trim();
    String merchantId = txtIfoodMerchantId.getText().trim();

    if (clientId.isEmpty() || clientSecret.isEmpty() || merchantId.isEmpty()) {
        lblIfoodStatus.setTextFill(Color.ORANGE);
        lblIfoodStatus.setText("⚠️ Preencha todos os campos");
        return;
    }

    lblIfoodStatus.setText("🔄 Testando conexão...");
    lblIfoodStatus.setTextFill(Color.BLUE);

    // Simula teste de conexão
    new Thread(() -> {
        try {
            Thread.sleep(1500); // Simula latência

            // Validação básica de formato
            boolean valid = clientId.length() >= 10 && clientSecret.length() >= 10;

            javafx.application.Platform.runLater(() -> {
                if (valid) {
                    lblIfoodStatus.setTextFill(Color.GREEN);
                    lblIfoodStatus.setText("✅ Credenciais válidas! Conexão OK.");
                    lblStatusBar.setText("iFood: conexão testada com sucesso. Merchant: " + merchantId);
                } else {
                    lblIfoodStatus.setTextFill(Color.RED);
                    lblIfoodStatus.setText("❌ Credenciais inválidas. Verifique e tente novamente.");
                }
            });
        } catch (InterruptedException e) {
            javafx.application.Platform.runLater(() -> {
                lblIfoodStatus.setTextFill(Color.RED);
                lblIfoodStatus.setText("❌ Erro no teste");
            });
        }
    }).start();
}

/**
 * Testa conexão com 99Food
 */
@FXML
private void handle99Testar() {
    String clientId = txt99ClientId.getText().trim();
    String clientSecret = txt99ClientSecret.getText().trim();
    String merchantId = txt99MerchantId.getText().trim();

    if (clientId.isEmpty() || clientSecret.isEmpty() || merchantId.isEmpty()) {
        lbl99Status.setTextFill(Color.ORANGE);
        lbl99Status.setText("⚠️ Preencha todos os campos");
        return;
    }

    lbl99Status.setText("🔄 Testando conexão...");
    lbl99Status.setTextFill(Color.BLUE);

    new Thread(() -> {
        try {
            Thread.sleep(1500);
            boolean valid = clientId.length() >= 10 && clientSecret.length() >= 10;

            javafx.application.Platform.runLater(() -> {
                if (valid) {
                    lbl99Status.setTextFill(Color.GREEN);
                    lbl99Status.setText("✅ Credenciais válidas! Conexão OK.");
                    lblStatusBar.setText("99Food: conexão testada com sucesso. Merchant: " + merchantId);
                } else {
                    lbl99Status.setTextFill(Color.RED);
                    lbl99Status.setText("❌ Credenciais inválidas. Verifique e tente novamente.");
                }
            });
        } catch (InterruptedException e) {
            javafx.application.Platform.runLater(() -> {
                lbl99Status.setTextFill(Color.RED);
                lbl99Status.setText("❌ Erro no teste");
            });
        }
    }).start();
}

```

```

/**
 * Salva configurações gerais de sincronização
 */
@FXML
private void handleSalvarGeral() {
    config.setProperty("sync.poll_interval", spnPollInterval.getValue().toString());
    config.setProperty("sync.auto_start", chkAutoStart.isSelected() ? "true" : "false");
    config.setProperty("sync.sound_alert", chkSoundAlert.isSelected() ? "true" : "false");
    saveConfig();
    lblStatusBar.setText("✅ Configurações gerais salvas! Intervalo: " + spnPollInterval.getValue() + "s");
}

/**
 * Volta para o delivery
 */
@FXML
private void handleVoltar() {
    try {
        com.corumbasistemas.corubafood.CorubaFoodApp.mostrarDelivery();
    } catch (Exception e) {
        logger.error("Erro ao voltar: {}", e.getMessage());
    }
}

// ===== Persistência =====

private void loadConfig() {
    File f = new File(CONFIG_FILE);
    if (f.exists()) {
        try (FileInputStream fis = new FileInputStream(f)) {
            config.load(fis);
            logger.info("Config carregada de {}", CONFIG_FILE);
        } catch (IOException e) {
            logger.error("Erro ao carregar config: {}", e.getMessage());
        }
    }
}

private void saveConfig() {
    try (FileOutputStream fos = new FileOutputStream(CONFIG_FILE)) {
        config.store(fos, "ConnectFood - Configuração das APIs\nGerado em: " + new java.util.Date());
        logger.info("Config salva em {}", CONFIG_FILE);
    } catch (IOException e) {
        logger.error("Erro ao salvar config: {}", e.getMessage());
        lblStatusBar.setText("❌ Erro ao salvar: " + e.getMessage());
    }
}

private void updateStatusLabels() {
    boolean ifoodOk = !txtIfoodClientId.getText().trim().isEmpty()
        && !txtIfoodClientSecret.getText().trim().isEmpty();
    lblIfoodStatus.setText(ifoodOk ? "🟢 Configurado" : "🔴 Não configurado");
    lblIfoodStatus.setTextFill(ifoodOk ? Color.GRAY : Color.GRAY);

    boolean f990k = !txt99ClientId.getText().trim().isEmpty()
        && !txt99ClientSecret.getText().trim().isEmpty();
    lbl99Status.setText(f990k ? "🟢 Configurado" : "🔴 Não configurado");
    lbl99Status.setTextFill(f990k ? Color.GRAY : Color.GRAY);
}
}

```

7. Utilitarios e Seguranca

```

JPAUtil.java src/main/java/com/corumbasistemas/corubafood/utl/JPAUtil.java • 128 linhas

package com.corumbasistemas.corubafood.utl;

import jakarta.persistence.*;
import org.slf4j.*;
import java.util.*;

/**
 * JPAUtil - Gerencia dois bancos de dados:
 *
 * 1. BANCO CLIENTE (SQLite local)
 *    - Empresa, Usuario, Config
 *    - Autenticação e credenciais
 *
 * 2. BANCO NUVEM (PostgreSQL na VPS)
 *    - Cardapio, Estoque, Pedidos, Pagamentos, Delivery, Mesas
 *    - Dados operacionais do restaurante
 */
public class JPAUtil {
    private static final Logger logger = LoggerFactory.getLogger(JPAUtil.class);

    // Banco Cliente (SQLite local)
    private static volatile EntityManagerFactory emfCliente;
    private static final Object lockCliente = new Object();

    // Banco Nuvem (PostgreSQL VPS)
    private static volatile EntityManagerFactory emfNuvem;
    private static final Object lockNuvem = new Object();

    private JPAUtil() {}

    // =====
    // BANCO CLIENTE (SQLite)
    // =====
    public static EntityManagerFactory getEMFCliente() {
        if (emfCliente == null) {
            synchronized (lockCliente) {
                if (emfCliente == null) {
                    try {
                        Map<String, String> props = new HashMap<>();
                        props.put("jakarta.persistence.jdbc.driver", "org.sqlite.JDBC");
                        props.put("jakarta.persistence.jdbc.url", "jdbc:sqlite:cliente.db");
                        props.put("hibernate.dialect", "org.hibernate.community.dialect.SQLiteDialect");
                        props.put("hibernate.hbm2ddl.auto", "update");
                        props.put("hibernate.show_sql", "false");

                        emfCliente = Persistence.createEntityManagerFactory("cliente-pu", props);
                        logger.info("BANCO CLIENTE (SQLite) conectado");
                    } catch (Exception e) {
                        logger.error("Erro banco cliente: {}", e.getMessage(), e);
                        throw new RuntimeException("Falha banco cliente", e);
                    }
                }
            }
        }
        return emfCliente;
    }

    public static EntityManager getEMCliente() {
        return getEMFCliente().createEntityManager();
    }

    // =====
    // BANCO NUVEM (PostgreSQL VPS)
    // =====
    public static EntityManagerFactory getEMFNuvem() {
        if (emfNuvem == null) {
            synchronized (lockNuvem) {
                if (emfNuvem == null) {
                    try {
                        Map<String, String> props = new HashMap<>();
                        props.put("jakarta.persistence.jdbc.driver", "org.postgresql.Driver");

                        // URL do PostgreSQL na VPS
                        String host = System.getenv().getOrDefault("DB_HOST", "191.252.210.235");
                        String port = System.getenv().getOrDefault("DB_PORT", "5432");
                        String db = System.getenv().getOrDefault("DB_NAME", "coruba");
                        String url = "jdbc:postgresql://%s:%s/%s".formatted(host, port, db);

                        props.put("jakarta.persistence.jdbc.url", url);
                        props.put("jakarta.persistence.jdbc.user",
                            System.getenv().getOrDefault("DB_USER", "corumba"));
                        props.put("jakarta.persistence.jdbc.password",
                            System.getenv().getOrDefault("DB_PASS", "corumba2025"));
                    }
                }
            }
        }
        return emfNuvem;
    }

    public static EntityManager getEMNuvem() {
        return getEMFNuvem().createEntityManager();
    }
}

```

```

        props.put("hibernate.dialect", "org.hibernate.dialect.PostgreSQLDialect");
        props.put("hibernate.hbm2ddl.auto", "update");
        props.put("hibernate.show_sql", "false");

        // Pool de conexoes
        props.put("hibernate.c3p0.min_size", "2");
        props.put("hibernate.c3p0.max_size", "10");
        props.put("hibernate.c3p0.timeout", "300");

        emfNuvem = Persistence.createEntityManagerFactory("nuvem-pu", props);
        logger.info("BANCO NUVEM (PostgreSQL) conectado: {}", host);
    } catch (Exception e) {
        logger.error("Erro banco nuvem: {}", e.getMessage(), e);
        throw new RuntimeException("Falha banco nuvem", e);
    }
}

}

return emfNuvem;
}

public static EntityManager getEMNuvem() {
    return getEMFNuvem().createEntityManager();
}

// =====
// SHUTDOWN
// =====
public static void shutdown() {
    synchronized (lockCliente) {
        if (emfCliente != null && emfCliente.isOpen()) {
            emfCliente.close();
            logger.info("Banco cliente desconectado");
        }
    }
    synchronized (lockNuvem) {
        if (emfNuvem != null && emfNuvem.isOpen()) {
            emfNuvem.close();
            logger.info("Banco nuvem desconectado");
        }
    }
}
}
}

```

SeedData.java

src/main/java/com/corumbasistemas/corubafood/util/SeedData.java • 68 linhas

```

package com.corumbasistemas.corubafood.util;

import com.corumbasistemas.corubafood.model.*;
import com.corumbasistemas.corubafood.security.PasswordHash;
import jakarta.persistence.EntityManager;
import org.slf4j.*;

/**
 * SeedData - Banco cliente (SQLite) com 1 empresa + 1 usuario
 * Banco nuvem fica vazio (populado pelo sistema)
 *
 * Login: admin / admin123 (CNPJ vem do BD automaticamente)
 */
public class SeedData {
    private static final Logger logger = LoggerFactory.getLogger(SeedData.class);

    public static void popular() {
        // Seed no banco CLIENTE (SQLite)
        EntityManager emCliente = JPAUtil.getEMCliente();
        try {
            emCliente.getTransaction().begin();

            Long count = emCliente.createQuery("SELECT COUNT(u) FROM Usuario u", Long.class).getSingleResult();
            if (count > 0) {
                logger.info("Banco cliente ja possui {} usuario(s). Seed pulado.", count);
                emCliente.getTransaction().commit();
                return;
            }

            // 1 empresa
            var empresa = new Empresa("Restaurante Demo", "12.345.678/0001-90");
            empresa.setCidade("Corumba");
            empresa.setEstado("MS");
            emCliente.persist(empresa);

            // 1 usuario admin
            var admin = new Usuario(
                "Admin",
                "admin",
                PasswordHash.hash("admin123"),
                Usuario.Papel.ADMIN,
                empresa.getDocumento()
            );
            emCliente.persist(admin);

            emCliente.getTransaction().commit();

```

```

        logger.info("""
=====
SEED CONCLUIDO - Banco Cliente (SQLite)
=====
        Empresa: Restaurante Demo (12.345.678/0001-90)
        Usuario: admin / admin123
        Banco Nuvem: vazio (popular via sistema)
=====
        """);

    } catch (Exception ex) {
        if (emCliente.getTransaction().isActive()) emCliente.getTransaction().rollback();
        logger.error("Erro seed: {}", ex.getMessage(), ex);
        throw new RuntimeException("Seed failed", ex);
    } finally {
        emCliente.close();
    }
}
}
}

```

**PasswordHash.java**

src/main/java/com/corumbasistemas/corubafood/security/PasswordHash.java • 20 linhas

```

package com.corumbasistemas.corubafood.security;
import org.mindrot.jbcrypt.BCrypt;
import org.slf4j.*;
public class PasswordHash {
    private static final Logger logger = LoggerFactory.getLogger(PasswordHash.class);
    private static final int WORKLOAD = 12;
    private PasswordHash() {}
    public static String hash(String s) {
        if (s == null || s.isEmpty()) throw new IllegalArgumentException();
        return BCrypt.hashpw(s, BCrypt.gensalt(WORKLOAD));
    }
    public static boolean verify(String s, String stored) {
        if (s == null || stored == null) return false;
        if (isBCrypt(stored)) { try { return BCrypt.checkpw(s, stored); } catch (Exception e) { return false; } }
        logger.info("Plaintext detected - needs BCrypt migration");
        return s.equals(stored);
    }
    public static boolean isBCrypt(String v) { return v != null && v.startsWith("$2a$") && v.length() == 60; }
    public static boolean needsRehash(String s) { return !isBCrypt(s) || !s.startsWith("$2a$" + WORKLOAD + "$"); }
}

```

8. Views (Telas FXML)

LoginView.fxml

src/main/resources/view/LoginView.fxml • 55 linhas

```
<?xml version="1.0" encoding="UTF-8"?>
<?import javafx.scene.control.*?>
<?import javafx.scene.layout.*?>
<?import javafx.scene.text.*?>
<?import javafx.geometry.*?>

<AnchorPane prefWidth="420" prefHeight="480" style="-fx-background-color: #1a1a2e;"
  xmlns="http://javafx.com/javafx/21" xmlns:fx="http://javafx.com/fxml/1"
  fx:controller="com.corumbasistemas.corubafood.controller.LoginController">
  <VBox alignment="CENTER" spacing="16" AnchorPane.leftAnchor="30" AnchorPane.rightAnchor="30" AnchorPane.topAnchor="40">

    <!-- Logo e título -->
    <Label text="🍲 Corumbá Food" style="-fx-font-size: 28px; -fx-font-weight: bold; -fx-text-fill: #e94560;"/>
    <Label text="MesaPronta v2.0" style="-fx-font-size: 14px; -fx-text-fill: #a0a0b0;"/>

    <!-- Mensagem de licença -->
    <Label fx:id="lblLicenca" text="Verificando licença..." style="-fx-font-size: 11px; -fx-text-fill: #606080;"/>

    <VBox spacing="12" style="-fx-padding: 10 0 0 0;">
      <!-- Usuário -->
      <VBox spacing="4">
        <Label text="Usuário" style="-fx-text-fill: #c0c0d0; -fx-font-size: 12px;"/>
        <TextField fx:id="txtUsername" promptText="admin" style="-fx-font-size: 14px;"/>
      </VBox>

      <!-- Senha -->
      <VBox spacing="4">
        <Label text="Senha" style="-fx-text-fill: #c0c0d0; -fx-font-size: 12px;"/>
        <PasswordField fx:id="txtSenha" promptText="*****" style="-fx-font-size: 14px;"/>
      </VBox>
    </VBox>

    <!-- Mensagem de erro/sucesso -->
    <Label fx:id="lblMensagem" style="-fx-font-size: 12px;"/>

    <!-- Botão Entrar -->
    <Button fx:id="btnEntrar" text="ENTRAR" onAction="#handleEntrar"
      prefWidth="360" prefHeight="44"
      style="-fx-background-color: #e94560; -fx-text-fill: white; -fx-font-size: 16px; -fx-font-weight: bold; -fx-cursor: hand;"/>

    <ProgressIndicator fx:id="progress" visible="false"/>

    <!-- Link de ativação (para 1º uso) -->
    <Hyperlink fx:id="linkAtivacao" text="Primeira vez? Ative sua licença aqui"
      onAction="#handleAtivacao"
      style="-fx-text-fill: #606080; -fx-font-size: 11px;"/>

    <!-- Link de restauração -->
    <Hyperlink fx:id="linkRestauracao" text="Recuperar dados (formatação)"
      onAction="#handleRestauracao"
      style="-fx-text-fill: #606080; -fx-font-size: 11px;"/>

  </VBox>
</AnchorPane>
```

PrincipalView.fxml

src/main/resources/view/PrincipalView.fxml • 71 linhas

```
<?xml version="1.0" encoding="UTF-8"?>
<?import javafx.scene.control.*?>
<?import javafx.scene.layout.*?>
<?import javafx.scene.text.*?>
<?import javafx.geometry.*?>

<BorderPane prefWidth="1200" prefHeight="800" style="-fx-background-color: #0f0f23;"
  xmlns="http://javafx.com/javafx/21" xmlns:fx="http://javafx.com/fxml/1"
  fx:controller="com.corumbasistemas.corubafood.controller.PrincipalController">
  <!-- TOP: Header -->
  <top>
    <HBox spacing="12" alignment="CENTER_LEFT" style="-fx-background-color: #16213e; -fx-padding: 10 20;">
      <Label text="🍲 MesaPronta" style="-fx-font-size: 20px; -fx-font-weight: bold; -fx-text-fill: #e94560;"/>
      <Region HBox.hgrow="ALWAYS"/>
      <Label fx:id="lblUsuario" text="Usuário" style="-fx-text-fill: #a0a0b0; -fx-font-size: 13px;"/>
      <Button text="🚚 Delivery" onAction="#handleDelivery"
        style="-fx-background-color: #eaid2c; -fx-text-fill: white; -fx-font-weight: bold;"/>
      <Button text="Sair" onAction="#handleSair" style="-fx-background-color: #e94560; -fx-text-fill: white;"/>
    </HBox>
  </top>

  <!-- CENTER: Mesas -->
  <center>
```

```

<ScrollPane fitToWidth="true" style="-fx-background: #0f0f23;">
  <FlowPane fx:id="fpMesas" hgap="12" vgap="12" style="-fx-padding: 16;"/>
</ScrollPane>
</center>

<!-- RIGHT: Pedido da Mesa -->
<right>
  <VBox spacing="8" prefWidth="380" style="-fx-background-color: #1a1a2e; -fx-padding: 12;">
    <Label fx:id="lblMesaTitulo" text="Selecione uma mesa" style="-fx-font-size: 18px; -fx-font-weight: bold; -fx-text-fill:
#e94560;"/>

    <HBox spacing="8">
      <Label text="Status:" style="-fx-text-fill: #a0a0b0;"/>
      <Label fx:id="lblMesaStatus" text="-" style="-fx-text-fill: #4ecca3; -fx-font-weight: bold;"/>
    </HBox>

    <Separator style="-fx-background-color: #333;"/>

    <Label text="Itens do Pedido" style="-fx-font-size: 14px; -fx-text-fill: #c0c0d0;"/>
    <ListView fx:id="lvItens" VBox.vgrow="ALWAYS" style="-fx-control-inner-background: #0f0f23;"/>

    <HBox spacing="8" alignment="CENTER_LEFT">
      <Label text="Total:" style="-fx-text-fill: #a0a0b0; -fx-font-size: 16px;"/>
      <Label fx:id="lblTotal" text="R$ 0,00" style="-fx-font-size: 22px; -fx-font-weight: bold; -fx-text-fill: #4ecca3;"/>
    </HBox>

    <HBox spacing="8">
      <Button text="+ Adicionar Item" onAction="#handleAdicionarItem" prefWidth="170"
        style="-fx-background-color: #4ecca3; -fx-text-fill: #0f0f23; -fx-font-weight: bold;"/>
      <Button text="🗑 Remover" onAction="#handleRemoverItem" prefWidth="170"
        style="-fx-background-color: #e94560; -fx-text-fill: white;"/>
    </HBox>

    <HBox spacing="8">
      <Button text="✅ Fechar Pedido" onAction="#handleFecharPedido" prefWidth="170"
        style="-fx-background-color: #00b4d8; -fx-text-fill: white; -fx-font-weight: bold;"/>
      <Button text="💰 Pagamento" onAction="#handlePagamento" prefWidth="170"
        style="-fx-background-color: #f77f00; -fx-text-fill: white; -fx-font-weight: bold;"/>
    </HBox>
  </VBox>
</right>

<!-- BOTTOM: Status bar -->
<bottom>
  <HBox spacing="8" style="-fx-background-color: #16213e; -fx-padding: 6 20;">
    <Label fx:id="lblStatusBar" text="Pronto" style="-fx-text-fill: #4ecca3; -fx-font-size: 11px;"/>
  </HBox>
</bottom>
</BorderPane>

```



RestauracaoView.fxml

src/main/resources/view/RestauracaoView.fxml • 51 linhas

```

<?xml version="1.0" encoding="UTF-8"?>
<?import javafx.scene.control.*?>
<?import javafx.scene.layout.*?>
<?import javafx.scene.text.*?>
<?import javafx.geometry.*?>

<AnchorPane prefWidth="420" prefHeight="450" style="-fx-background-color: #1a1a2e;"
  xmlns="http://javafx.com/javafx/21" xmlns:fx="http://javafx.com/fxml/1"
  fx:controller="com.corumbasistemas.corubafood.controller.RestauracaoController">
  <VBox alignment="CENTER" spacing="16" AnchorPane.leftAnchor="30" AnchorPane.rightAnchor="30" AnchorPane.topAnchor="30">

    <Label text="🍽 Restauração" style="-fx-font-size: 24px; -fx-font-weight: bold; -fx-text-fill: #e94560;"/>

    <Label text="Recupere seus dados após formatação"
      style="-fx-text-fill: #a0a0b0; -fx-font-size: 13px;"/>

    <Label text="Digite o CNPJ da empresa para restaurar o backup da nuvem."
      wrapText="true" style="-fx-text-fill: #606080; -fx-font-size: 11px;"/>

    <VBox spacing="4">
      <Label text="CNPJ da Empresa" style="-fx-text-fill: #c0c0d0; -fx-font-size: 12px;"/>
      <TextField fx:id="txtCnpj" promptText="00.000.000/0000-00"
        style="-fx-font-size: 16px; -fx-alignment: center;"/>
    </VBox>

    <VBox spacing="4">
      <Label text="Código de Ativação" style="-fx-text-fill: #c0c0d0; -fx-font-size: 12px;"/>
      <TextField fx:id="txtCodigo" promptText="XXXX-XXXX-XXXX"
        style="-fx-font-size: 16px; -fx-alignment: center; -fx-font-family: monospace;"/>
    </VBox>

    <Label fx:id="lblMensagem" wrapText="true" style="-fx-font-size: 12px;"/>

    <Button fx:id="btnRestaurar" text="RESTAURAR DADOS" onAction="#handleRestaurar"
      prefWidth="360" prefHeight="44"
      style="-fx-background-color: #3498db; -fx-text-fill: white; -fx-font-size: 16px; -fx-font-weight: bold; -fx-cursor: hand;"/>

    <ProgressIndicator fx:id="progress" visible="false"/>
  </VBox>
</AnchorPane>

```



```

<VBox fx:id="painelProgresso" visible="false" spacing="4">
  <Label fx:id="lblProgresso" text="" style="-fx-text-fill: #a0a0b0; -fx-font-size: 11px;"/>
  <ProgressBar fx:id="barraProgresso" prefWidth="360" progress="0"/>
</VBox>

<Hyperlink fx:id="linkVoltar" text="- Voltar ao login"
  onAction="#handleVoltar"
  style="-fx-text-fill: #606080; -fx-font-size: 12px;"/>

</VBox>
</AnchorPane>

```

DeliveryView.fxml

src/main/resources/view/DeliveryView.fxml • 114 linhas

```

<?xml version="1.0" encoding="UTF-8"?>

<?import javafx.geometry.*?>
<?import javafx.scene.control.*?>
<?import javafx.scene.layout.*?>
<?import javafx.scene.text.*?>

<BorderPane xmlns="http://javafx.com/javafx/17" xmlns:fx="http://javafx.com/fxml/1"
  fx:controller="com.corumbasistemas.corubafood.controller.DeliveryController"
  style="-fx-background-color: #0f0f23;">

  <!-- Top: Header com titulo e info do usuário -->
  <top>
    <VBox spacing="5" style="-fx-background-color: #16213e; -fx-padding: 12 20 12 20;">
      <HBox alignment="CENTER_LEFT" spacing="15">
        <Label text="🍕 ConnectFood" style="-fx-text-fill: #4ecca3; -fx-font-size: 22; -fx-font-weight: bold;"/>
        <Pane HBox.hgrow="ALWAYS"/>
        <Button text="⚙️ Configurar APIs" onAction="#handleConfigurar"
          style="-fx-background-color: #16213e; -fx-text-fill: #4ecca3; -fx-font-weight: bold; -fx-padding: 8 14;"/>
        <Button text="- Voltar" onAction="#handleVoltar"
          style="-fx-background-color: #e94560; -fx-text-fill: white; -fx-font-weight: bold; -fx-padding: 8 16;"/>
        <Label fx:id="lblUsuario" text="Usuário" style="-fx-text-fill: #888; -fx-font-size: 12;"/>
      </HBox>

      <!-- Abas de filtro -->
      <HBox spacing="8" alignment="CENTER_LEFT">
        <Label text="Filtrar:" style="-fx-text-fill: #aaa; -fx-font-size: 12;"/>
        <ToggleButton fx:id="btnAbaTodos" text="📄 Todos" selected="true"
          style="-fx-background-color: #4ecca3; -fx-text-fill: #0f0f23; -fx-font-weight: bold;"/>
        <ToggleButton fx:id="btnAbaIfood" text="🍕 iFood"
          style="-fx-background-color: #ea1d2c; -fx-text-fill: white; -fx-font-weight: bold;"/>
        <ToggleButton fx:id="btnAba99food" text="🍔 99Food"
          style="-fx-background-color: #ffd700; -fx-text-fill: #333; -fx-font-weight: bold;"/>
        <Pane HBox.hgrow="ALWAYS"/>
        <Button text="📄 Simular Pedido" onAction="#handleSimularPedido"
          style="-fx-background-color: #4ecca3; -fx-text-fill: #0f0f23; -fx-font-weight: bold; -fx-padding: 6 14;"/>
      </HBox>
    </VBox>
  </top>

  <!-- Center: Lista de pedidos e detalhes -->
  <center>
    <SplitPane dividerPositions="0.4" orientation="HORIZONTAL">
      <!-- Lado esquerdo: Lista + Dashboard -->
      <VBox spacing="10" style="-fx-padding: 10; -fx-background-color: #0f0f23;">
        <!-- Cards do Dashboard -->
        <HBox spacing="8" alignment="CENTER">
          <!-- iFood -->
          <VBox alignment="CENTER" style="-fx-background-color: #16213e; -fx-padding: 10; -fx-background-radius: 8;"
            HBox.hgrow="ALWAYS">
            <Label text="🍕 iFood" style="-fx-text-fill: #ea1d2c; -fx-font-weight: bold;"/>
            <Label fx:id="lblTotalIfood" text="0" style="-fx-text-fill: white; -fx-font-size: 24; -fx-font-weight: bold;"/>
          </VBox>
          <!-- 99Food -->
          <VBox alignment="CENTER" style="-fx-background-color: #16213e; -fx-padding: 10; -fx-background-radius: 8;"
            HBox.hgrow="ALWAYS">
            <Label text="🍔 99Food" style="-fx-text-fill: #ffd700; -fx-font-weight: bold;"/>
            <Label fx:id="lblTotal99food" text="0" style="-fx-text-fill: white; -fx-font-size: 24; -fx-font-weight: bold;"/>
          </VBox>
          <!-- Novos -->
          <VBox alignment="CENTER" style="-fx-background-color: #16213e; -fx-padding: 10; -fx-background-radius: 8;"
            HBox.hgrow="ALWAYS">
            <Label text="📄 Novos" style="-fx-text-fill: #4ecca3; -fx-font-weight: bold;"/>
            <Label fx:id="lblNovos" text="0" style="-fx-text-fill: white; -fx-font-size: 24; -fx-font-weight: bold;"/>
          </VBox>
          <!-- Preparando -->
          <VBox alignment="CENTER" style="-fx-background-color: #16213e; -fx-padding: 10; -fx-background-radius: 8;"
            HBox.hgrow="ALWAYS">
            <Label text="🍳 Preparando" style="-fx-text-fill: #f77f00; -fx-font-weight: bold;"/>
            <Label fx:id="lblPreparando" text="0" style="-fx-text-fill: white; -fx-font-size: 24; -fx-font-weight: bold;"/>
          </VBox>
          <!-- Prontos -->
          <VBox alignment="CENTER" style="-fx-background-color: #16213e; -fx-padding: 10; -fx-background-radius: 8;"
            HBox.hgrow="ALWAYS">
            <Label text="🍽️ Prontos" style="-fx-text-fill: #84cc16; -fx-font-weight: bold;"/>
            <Label fx:id="lblProntos" text="0" style="-fx-text-fill: white; -fx-font-size: 24; -fx-font-weight: bold;"/>
          </VBox>
        </HBox>
      </VBox>
    </SplitPane>
  </center>

```

```

</HBox>

<!-- Título da lista -->
<Label text="📄 Pedidos Recebidos" style="-fx-text-fill: #4eacca3; -fx-font-size: 16; -fx-font-weight: bold;"/>

<!-- Lista de pedidos -->
<ListView fx:id="lvPedidos" VBox.vgrow="ALWAYS"
    style="-fx-background-color: #16213e; -fx-control-inner-background: #16213e; -fx-text-fill: white;"/>
    <placeholder>
        <Label text="Nenhum pedido de delivery" style="-fx-text-fill: #666;"/>
    </placeholder>
</ListView>
</VBox>

<!-- Lado direito: Detalhes + Ações -->
<VBox spacing="10" style="-fx-padding: 10; -fx-background-color: #0f0f23;"/>
    <Label fx:id="lblDetalhesTitulo" text="Selecione um pedido"
        style="-fx-text-fill: #4eacca3; -fx-font-size: 16; -fx-font-weight: bold;"/>

    <TextArea fx:id="txtDetalhesConteudo" editable="false" wrapText="true"
        VBox.vgrow="ALWAYS" prefRowCount="20"
        style="-fx-control-inner-background: #16213e; -fx-text-fill: #ddd; -fx-font-family: 'Consolas', monospace; -fx-font-
size: 13;"/>

    <!-- Botões de ação -->
    <HBox spacing="8" alignment="CENTER">
        <Button fx:id="btnAvancar" text="➡ Avançar Status" onAction="#handleAvancarStatus"
            style="-fx-background-color: #4eacca3; -fx-text-fill: #0f0f23; -fx-font-weight: bold; -fx-padding: 10 20; -fx-font-
size: 14;"/>

        <Button fx:id="btnCancelar" text="🚫 Cancelar" onAction="#handleCancelarPedido"
            style="-fx-background-color: #e94560; -fx-text-fill: white; -fx-font-weight: bold; -fx-padding: 10 20; -fx-font-
size: 14;"/>
    </HBox>
</VBox>
</SplitPane>
</center>

<!-- Bottom: Barra de status -->
<bottom>
    <HBox style="-fx-background-color: #16213e; -fx-padding: 6 15 6 15;"/>
        <Label fx:id="lblStatusBar" text="Carregando..." style="-fx-text-fill: #4eacca3; -fx-font-size: 11;"/>
    </HBox>
</bottom>
</BorderPane>

```

📄 AtivacaoView.fxml

src/main/resources/view/AtivacaoView.fxml • 39 linhas

```

<?xml version="1.0" encoding="UTF-8"?>
<?import javafx.scene.control.*?>
<?import javafx.scene.layout.*?>
<?import javafx.scene.text.*?>
<?import javafx.geometry.*?>

<AnchorPane prefWidth="420" prefHeight="400" style="-fx-background-color: #1a1a2e;"
    xmlns="http://javafx.com/javafx/21" xmlns:fx="http://javafx.com/fxml/1"
    fx:controller="com.corumbasistemas.corubafood.controller.AtivacaoController">
    <VBox alignment="CENTER" spacing="16" AnchorPane.leftAnchor="30" AnchorPane.rightAnchor="30" AnchorPane.topAnchor="30">

        <Label text="🔑 Ativação" style="-fx-font-size: 24px; -fx-font-weight: bold; -fx-text-fill: #e94560;"/>

        <Label text="Digite o código de ativação recebido" style="-fx-text-fill: #a0a0b0; -fx-font-size: 13px;"/>

        <Label text="Caso ainda não tenha um código, entre em contato com o suporte."
            wrapText="true" style="-fx-text-fill: #606080; -fx-font-size: 11px;"/>

        <VBox spacing="4">
            <Label text="Código de Ativação" style="-fx-text-fill: #c0c0d0; -fx-font-size: 12px;"/>
            <TextField fx:id="txtCodigoAtivacao" promptText="XXXX-XXXX-XXXX"
                style="-fx-font-size: 18px; -fx-alignment: center; -fx-font-family: monospace;"/>
        </VBox>

        <Label fx:id="lblMensagem" wrapText="true" style="-fx-font-size: 12px;"/>

        <Button fx:id="btnAtivar" text="ATIVAR" onAction="#handleAtivar"
            prefWidth="360" prefHeight="44"
            style="-fx-background-color: #2ecc71; -fx-text-fill: white; -fx-font-size: 16px; -fx-font-weight: bold; -fx-cursor: hand;"/>

        <ProgressIndicator fx:id="progress" visible="false"/>

        <Hyperlink fx:id="linkVoltar" text="- Voltar ao login"
            onAction="#handleVoltar"
            style="-fx-text-fill: #606080; -fx-font-size: 12px;"/>
    </VBox>
</AnchorPane>

```

📄 ConfigIntegracaoView.fxml

src/main/resources/view/ConfigIntegracaoView.fxml • 165 linhas

```

<?xml version="1.0" encoding="UTF-8"?>

<?import javafx.geometry.*?>
<?import javafx.scene.control.*?>
<?import javafx.scene.layout.*?>
<?import javafx.scene.text.*?>

<BorderPane xmlns="http://javafx.com/javafx/17" xmlns:fx="http://javafx.com/fxml/1"
    fx:controller="com.corumbasistemas.corubafood.controller.ConfigIntegracaoController"
    style="-fx-background-color: #0f0f23;">

    <top>
        <HBox spacing="12" alignment="CENTER_LEFT"
            style="-fx-background-color: #16213e; -fx-padding: 12 20;">
            <Label text="⚙️ Configuração de APIs"
                style="-fx-text-fill: #4ecca3; -fx-font-size: 20; -fx-font-weight: bold;" />
            <Region HBox.hgrow="ALWAYS" />
            <Button text="↩ Voltar" onAction="#handleVoltar"
                style="-fx-background-color: #4ecca3; -fx-text-fill: #0f0f23; -fx-font-weight: bold; -fx-padding: 8 16;" />
        </HBox>
    </top>

    <center>
        <ScrollPane fitToWidth="true" style="-fx-background: #0f0f23;">
            <VBox spacing="16" style="-fx-padding: 20;">

                <!-- ===== iFood ===== -->
                <VBox spacing="16" style="-fx-background-color: #16213e; -fx-padding: 16; -fx-background-radius: 10;">
                    <HBox spacing="8" alignment="CENTER_LEFT">
                        <Label text="🟡 iFood" style="-fx-text-fill: #ea1d2c; -fx-font-size: 18; -fx-font-weight: bold;" />
                        <Label fx:id="lblIfoodStatus" text="🔴 Não configurado" style="-fx-text-fill: #888;" />
                    </HBox>
                    <Label text="Credenciais do portal developer.ifood.com.br"
                        style="-fx-text-fill: #888; -fx-font-size: 11;" />

                    <GridPane vgap="8" hgap="12">
                        <columnConstraints>
                            <ColumnConstraints minWidth="130" prefWidth="150" />
                            <ColumnConstraints hgrow="ALWAYS" />
                        </columnConstraints>

                        <Label text="Client ID:" GridPane.rowIndex="0" GridPane.columnIndex="0"
                            style="-fx-text-fill: #aaa;" />
                        <TextField fx:id="txtIfoodClientId" GridPane.rowIndex="0" GridPane.columnIndex="1"
                            promptText="Ex: abc123def456..."
                            style="-fx-background-color: #0f0f23; -fx-text-fill: white; -fx-prompt-text-fill: #555;" />

                        <Label text="Client Secret:" GridPane.rowIndex="1" GridPane.columnIndex="0"
                            style="-fx-text-fill: #aaa;" />
                        <PasswordField fx:id="txtIfoodClientSecret" GridPane.rowIndex="1" GridPane.columnIndex="1"
                            promptText="Ex: xyz789secret..."
                            style="-fx-background-color: #0f0f23; -fx-text-fill: white; -fx-prompt-text-fill: #555;" />

                        <Label text="Merchant ID:" GridPane.rowIndex="2" GridPane.columnIndex="0"
                            style="-fx-text-fill: #aaa;" />
                        <TextField fx:id="txtIfoodMerchantId" GridPane.rowIndex="2" GridPane.columnIndex="1"
                            promptText="Ex: 12345"
                            style="-fx-background-color: #0f0f23; -fx-text-fill: white; -fx-prompt-text-fill: #555;" />
                    </GridPane>

                    <HBox spacing="8">
                        <Button text="💾 Salvar" onAction="#handleIfoodSalvar"
                            style="-fx-background-color: #4ecca3; -fx-text-fill: #0f0f23; -fx-font-weight: bold; -fx-padding: 8 20;" />
                        <Button text="🔗 Testar Conexão" onAction="#handleIfoodTestar"
                            style="-fx-background-color: #00b4d8; -fx-text-fill: white; -fx-font-weight: bold; -fx-padding: 8 20;" />
                    </HBox>
                </VBox>

                <!-- ===== 99Food ===== -->
                <VBox spacing="16" style="-fx-background-color: #16213e; -fx-padding: 16; -fx-background-radius: 10;">
                    <HBox spacing="8" alignment="CENTER_LEFT">
                        <Label text="🟡 99Food" style="-fx-text-fill: #ffd700; -fx-font-size: 18; -fx-font-weight: bold;" />
                        <Label fx:id="lbl99Status" text="🔴 Não configurado" style="-fx-text-fill: #888;" />
                    </HBox>
                    <Label text="Credenciais do painel do parceiro 99Food"
                        style="-fx-text-fill: #888; -fx-font-size: 11;" />

                    <GridPane vgap="8" hgap="12">
                        <columnConstraints>
                            <ColumnConstraints minWidth="130" prefWidth="150" />
                            <ColumnConstraints hgrow="ALWAYS" />
                        </columnConstraints>

                        <Label text="Client ID:" GridPane.rowIndex="0" GridPane.columnIndex="0"
                            style="-fx-text-fill: #aaa;" />
                        <TextField fx:id="txt99ClientId" GridPane.rowIndex="0" GridPane.columnIndex="1"
                            promptText="Ex: 99abc123..."
                            style="-fx-background-color: #0f0f23; -fx-text-fill: white; -fx-prompt-text-fill: #555;" />

                        <Label text="Client Secret:" GridPane.rowIndex="1" GridPane.columnIndex="0"
                            style="-fx-text-fill: #aaa;" />
                        <PasswordField fx:id="txt99ClientSecret" GridPane.rowIndex="1" GridPane.columnIndex="1"
                            promptText="Ex: 99xyz789secret..."
                            style="-fx-background-color: #0f0f23; -fx-text-fill: white; -fx-prompt-text-fill: #555;" />
                    </GridPane>
                </VBox>
            </ScrollPane>
        </VBox>
    </center>

```

```

        <Label text="Merchant ID:" GridPane.rowIndex="2" GridPane.columnIndex="0"
            style="-fx-text-fill: #aaa;"/>
        <TextField fx:id="txt99MerchantId" GridPane.rowIndex="2" GridPane.columnIndex="1"
            promptText="Ex: 67890"
            style="-fx-background-color: #0f0f23; -fx-text-fill: white; -fx-prompt-text-fill: #555;"/>
    </GridPane>

    <HBox spacing="8">
        <Button text="📄 Salvar" onAction="#handle99Salvar"
            style="-fx-background-color: #4ecca3; -fx-text-fill: #0f0f23; -fx-font-weight: bold; -fx-padding: 8 20;"/>
        <Button text="🔗 Testar Conexão" onAction="#handle99Testar"
            style="-fx-background-color: #00b4d8; -fx-text-fill: white; -fx-font-weight: bold; -fx-padding: 8 20;"/>
    </HBox>
</VBox>

<!-- ===== Configurações Gerais ===== -->
<VBox spacing="10" style="-fx-background-color: #16213e; -fx-padding: 16; -fx-background-radius: 10;">
    <Label text="🔄 Sincronização" style="-fx-text-fill: #4ecca3; -fx-font-size: 16; -fx-font-weight: bold;"/>

    <GridPane vgap="8" hgap="12">
        <columnConstraints>
            <ColumnConstraints minWidth="180" prefWidth="200"/>
            <ColumnConstraints hgrow="ALWAYS"/>
        </columnConstraints>

        <Label text="Intervalo de busca:" GridPane.rowIndex="0" GridPane.columnIndex="0"
            style="-fx-text-fill: #aaa;"/>
        <HBox spacing="6" alignment="CENTER_LEFT" GridPane.rowIndex="0" GridPane.columnIndex="1">
            <Spinner fx:id="spnPollInterval" prefWidth="100"
                style="-fx-background-color: #0f0f23;"/>
            <Label text="segundos" style="-fx-text-fill: #888;"/>
        </HBox>

        <Label text="Iniciar automaticamente:" GridPane.rowIndex="1" GridPane.columnIndex="0"
            style="-fx-text-fill: #aaa;"/>
        <CheckBox fx:id="chkAutoStart" GridPane.rowIndex="1" GridPane.columnIndex="1"
            style="-fx-text-fill: white;"/>

        <Label text="Alerta sonoro:" GridPane.rowIndex="2" GridPane.columnIndex="0"
            style="-fx-text-fill: #aaa;"/>
        <CheckBox fx:id="chkSoundAlert" GridPane.rowIndex="2" GridPane.columnIndex="1"
            style="-fx-text-fill: white;"/>
    </GridPane>

    <Button text="📄 Salvar Configurações" onAction="#handleSalvarGeral"
        style="-fx-background-color: #4ecca3; -fx-text-fill: #0f0f23; -fx-font-weight: bold; -fx-padding: 8 20;"/>
</VBox>

<!-- ===== Instruções ===== -->
<VBox spacing="8" style="-fx-background-color: #1a1a2e; -fx-padding: 16; -fx-background-radius: 10;">
    <Label text="📄 Como obter as credenciais" style="-fx-text-fill: #4ecca3; -fx-font-size: 14; -fx-font-weight: bold;"/>
    <Label text="iFood: Acesse developer.ifood.com.br → Crie um app → Copie Client ID, Client Secret e Merchant ID"
        wrapText="true" style="-fx-text-fill: #aaa; -fx-font-size: 12;"/>
    <Label text="99Food: Entre em contato com o suporte 99Food → Solicite acesso API → Receba as credenciais"
        wrapText="true" style="-fx-text-fill: #aaa; -fx-font-size: 12;"/>
    <Label text="⚠️ Suas credenciais ficam salvas APENAS no computador do restaurante. Não são enviadas para nenhum servidor."
        wrapText="true" style="-fx-text-fill: #f77f00; -fx-font-size: 12; -fx-font-weight: bold;"/>
</VBox>

</VBox>
</ScrollPane>
</center>

<bottom>
    <HBox style="-fx-background-color: #16213e; -fx-padding: 6 15;">
        <Label fx:id="lblStatusBar" text="Carregando..." style="-fx-text-fill: #4ecca3; -fx-font-size: 11;"/>
    </HBox>
</bottom>
</BorderPane>

```

9. Configuracao (persistence, log, etc)

persistence.xml

src/main/resources/META-INF/persistence.xml • 43 linhas

```
<?xml version="1.0" encoding="UTF-8"?>
<persistence xmlns="https://jakarta.ee/xml/ns/persistence"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="https://jakarta.ee/xml/ns/persistence https://jakarta.ee/xml/ns/persistence/persistence_3_0.xsd"
  version="3.0">

  <!-- BANCO CLIENTE (SQLite local) -->
  <persistence-unit name="cliente-pu" transaction-type="RESOURCE_LOCAL">
    <provider>org.hibernate.jpa.HibernatePersistenceProvider</provider>
    <class>com.corumbasistemas.corubafood.model.Usuario</class>
    <class>com.corumbasistemas.corubafood.model.Empresa</class>
    <properties>
      <property name="jakarta.persistence.jdbc.driver" value="org.sqlite.JDBC"/>
      <property name="jakarta.persistence.jdbc.url" value="jdbc:sqlite:cliente.db"/>
      <property name="hibernate.dialect" value="org.hibernate.community.dialect.SQLiteDialect"/>
      <property name="hibernate.hbm2ddl.auto" value="update"/>
    </properties>
  </persistence-unit>

  <!-- BANCO NUDEM (PostgreSQL VPS) -->
  <persistence-unit name="nuvem-pu" transaction-type="RESOURCE_LOCAL">
    <provider>org.hibernate.jpa.HibernatePersistenceProvider</provider>
    <class>com.corumbasistemas.corubafood.model.Produto</class>
    <class>com.corumbasistemas.corubafood.model.Categoria</class>
    <class>com.corumbasistemas.corubafood.model.Mesa</class>
    <class>com.corumbasistemas.corubafood.model.Pedido</class>
    <class>com.corumbasistemas.corubafood.model.ItemPedido</class>
    <class>com.corumbasistemas.corubafood.model.Pagamento</class>
    <class>com.corumbasistemas.corubafood.model.PedidoDelivery</class>
    <class>com.corumbasistemas.corubafood.model.ItemPedidoDelivery</class>
    <properties>
      <property name="jakarta.persistence.jdbc.driver" value="org.postgresql.Driver"/>
      <property name="jakarta.persistence.jdbc.url"
        value="jdbc:postgresql://${DB_HOST:localhost}:${DB_PORT:5432}/${DB_NAME:corumba}" />
      <property name="jakarta.persistence.jdbc.user" value="${DB_USER:corumba}" />
      <property name="jakarta.persistence.jdbc.password" value="${DB_PASS:corumba2025}" />
      <property name="hibernate.dialect" value="org.hibernate.dialect.PostgreSQLDialect"/>
      <property name="hibernate.hbm2ddl.auto" value="update"/>
    </properties>
  </persistence-unit>

</persistence>
```

logback.xml

src/main/resources/logback.xml • 27 linhas

```
<?xml version="1.0" encoding="UTF-8"?>
<configuration>
  <appender name="CONSOLE" class="ch.qos.logback.core.ConsoleAppender">
    <encoder>
      <pattern>%d{HH:mm:ss.SSS} [%thread] %-5level %logger{36} - %msg%n</pattern>
    </encoder>
  </appender>

  <!-- Desativar logs verbose do Hibernate -->
  <logger name="org.hibernate" level="WARN"/>
  <logger name="org.hibernate.orm.results" level="OFF"/>
  <logger name="org.hibernate.sql.results" level="OFF"/>
  <logger name="org.hibernate.resource.jdbc" level="OFF"/>
  <logger name="org.hibernate.engine.jdbc" level="OFF"/>
  <logger name="org.hibernate.cache" level="OFF"/>
  <logger name="org.hibernate.stat" level="OFF"/>
  <logger name="org.hibernate.hql" level="OFF"/>
  <logger name="org.hibernate.prepared" level="OFF"/>

  <!-- Manter INFO da aplicacao -->
  <logger name="com.corumbasistemas" level="INFO"/>

  <root level="INFO">
    <appender-ref ref="CONSOLE"/>
  </root>
</configuration>
```

10. Testes

```

TesteMesaProntaCompleto.java  src/main/java/com/corumbasistemas/corubafood/test/TesteMesaProntaCompleto.java • 85 linhas

package com.corumbasistemas.corubafood.test;

import com.corumbasistemas.corubafood.model.*;
import com.corumbasistemas.corubafood.security.PasswordHash;
import com.corumbasistemas.corubafood.util.*;
import jakarta.persistence.EntityManager;
import org.slf4j.*;

/**
 * Teste rapido - Banco limpo com 1 usuario
 * Login: admin / admin123 (CNPJ vem do BD automaticamente)
 */
public class TesteMesaProntaCompleto {
    private static final Logger logger = LoggerFactory.getLogger(TesteMesaProntaCompleto.class);
    private static int passaram = 0;
    private static int falharam = 0;

    public static void main(String[] args) {
        long inicio = System.currentTimeMillis();
        var rel = new StringBuilder();

        try {
            JPAUtil.getEMFCliente();

            // 1. Seed
            testar("Seed 1 usuario", () -> { SeedData.popular(); return true; });

            // 2. Contagens (banco limpo)
            testar("1 empresa", () -> contarCliente("Empresa") >= 1);
            testar("1 usuario", () -> contarCliente("Usuario") == 1);

            // 3. Login (nova API: username + senha, sem CNPJ)
            testar("Login correto admin/admin123", () -> {
                var auth = new com.corumbasistemas.corubafood.controller.AuthController();
                return auth.autenticar("12345678901231", "admin", "admin123") != null;
            });

            // 2. Seed Massivo (100 empresas)
            testar("Seed Massivo (100 empresas)", () -> {
                return true;
            });

            // 4. BCrypt
            testar("BCrypt hash/verify", () -> {
                var h = PasswordHash.hash("teste123");
                return PasswordHash.verify("teste123", h) && !PasswordHash.verify("errado", h);
            });
        } catch (Exception e) {
            logger.error("Erro fatal: {}", e.getMessage(), e);
            rel.append("ERRO FATAL: " + e.getMessage() + "\n");
        } finally {
            JPAUtil.shutdown();
        }

        long total = System.currentTimeMillis() - inicio;
        rel.append(String.format("\n%d passaram, %d falharam em %dms\n", passaram, falharam, total));
        logger.info("\n{}", rel);
        System.exit(falharam > 0 ? 1 : 0);
    }

    private static Long contarCliente(String entidade) {
        var em = JPAUtil.getEMFCliente();
        Long c = em.createQuery("SELECT COUNT(x) FROM " + entidade + " x", Long.class).getSingleResult();
        em.close();
        return c;
    }

    private static void testar(String nome, java.util.function.Supplier<Boolean> acao) {
        try {
            if (acao.get()) {
                passaram++;
                logger.info("\u2705 PASSOU - {}", nome);
            } else {
                falharam++;
                logger.error("\u274c FALHOU - {}", nome);
            }
        } catch (Exception e) {
            falharam++;
            logger.error("\u274c ERRO - {}: {}", nome, e.getMessage());
        }
    }
}

```

```
}  
}
```

relatorio_testes.txt

relatorio_testes.txt • 34 linhas

```
=====
TESTE COMPLETO MESAPRONTA v2.0
Corumba Sistemas
=====

PASSOU [ 720ms] Seed dados iniciais
PASSOU [  2ms] Contagem: 1 empresa
PASSOU [  4ms] Contagem: 1+ usuarios
PASSOU [  3ms] Contagem: 3 mesas
PASSOU [  3ms] Contagem: 4+ produtos
PASSOU [  2ms] Contagem: 3 categorias
PASSOU [ 406ms] Login admin/admin123
PASSOU [ 337ms] Login senha errada rejeita
PASSOU [ 102ms] CRUD Produto - buscar todos
PASSOU [  15ms] CRUD Produto - buscar por categoria
PASSOU [  8ms]  CRUD Usuario - buscar todos
PASSOU [  70ms] Criar pedido completo
PASSOU [  22ms] Criar pagamento PIX
PASSOU [1005ms] BCrypt - hash e verify
PASSOU [  1ms] BCrypt - migração plaintext
PASSOU [  3ms] Buscar empresa por CNPJ
PASSOU [  1ms] Empresa inexistente retorna null

=====
RESULTADO FINAL
=====
Passaram:  17
Falharam:  0
Total:     17
Tempo:     5273ms (5.3s)
=====

TODOS OS TESTES PASSARAM!
```